Predicting SME Loan Default Risk in Kenya Using Gradient Boosting & Explainable AI

👥 GROUP MEMBERS

- **SHARON NAMUKI**
- **MALVIN CHEPTOO**
- **ABDULLAHI KULEI**

1. Business Understanding

1.1 Problem Context

Small and medium enterprises (SMEs) and youth entrepreneurs in emerging markets face limited access to affordable credit due to high perceived default risk. Manual credit assessments are slow, subjective, and difficult to scale.

This project develops a **supervised machine learning model** to **predict early loan default risk**, enabling lenders to:

```
Reduce non-performing loans
Intervene early with high-risk borrowers
Promote financial inclusion through fairer credit decisions
```

1.2 Stakeholders

Primary: Credit officers, risk analysts, loan managers

Secondary: SME lenders, fintech product teams, regulators

1.3 Objective

Can we predict whether a loan will default using borrower demographics, loan terms, and early repayment behavior?

```
Target variable: is_default (0 = non-default, 1 = default)

Prediction type: Binary classification/Multi_Class

Prediction timing: Early lifecycle
```

2. Environment Setup

In [1]:
```
# Our core library
import pandas as pd
import numpy as np
import joblib
from pathlib import Path

#visualization libraries
import seaborn as sns
import matplotlib.pyplot as plt
import warnings
warnings.filterwarnings('ignore')

# Setting style and color scheme
plt.style.use('seaborn-v0_8-darkgrid')
sns.set_palette("husl")

COLORS = {
    'default': ['#FF6B6B', '#4ECDC4'],
    'sequential': ['#1A2980', '#26D0CE'],
    'categorical': ['#FF9A76', '#FFEAA7', '#BDC581', '#4A69BB', '#6A89CC']
}

#Preprocessing
from sklearn.model_selection import train_test_split,StratifiedKFold,cross_val_s
from sklearn.preprocessing import MinMaxScaler, StandardScaler, OneHotEncoder,Rob
from sklearn.linear_model import LogisticRegression
from sklearn.ensemble import RandomForestClassifier
from sklearn.metrics import accuracy_score,precision_score,recall_score,f1_score,
from statsmodels.stats.outliers_influence import variance_inflation_factor
from sklearn.utils.class_weight import compute_class_weight
from sklearn.feature_selection import VarianceThreshold
from sklearn.base import BaseEstimator, TransformerMixin
import lightgbm as lgb



#Oversampling
from imblearn.over_sampling import SMOTE
from scipy import stats
import shap
```

3. Data Understanding

3.1 Data Overview

The dataset used in this analysis consists of **277,159 loan records** collected from **Micromart Africa's digital lending platform** for the purpose of **loan default prediction**. It contains several key feature groups, including: **borrower attributes** (e.g., age, gender, and account age), **loan**

characteristics (e.g., principal, interest, total loan amount, loan term, product type, and number of installments), and **repayment behavior indicators** (e.g., total amount paid, unpaid balance, days late, and numbers of late and on-time payments).

The dataset is structured as a **binary classification problem**, where the target variable indicates whether a borrower defaulted on a loan. It combines demographic, transactional, and behavioral information, enabling the development of supervised machine learning models for credit risk assessment.

3.2 Data Description

3.2.1 Importing the dataset

In [2]:
```
#importing and printing head
data = pd.read_csv("data/loan_dataset.csv")
data.head()
```

Out[2]:

	LoanId	BorrowerId	BorrowDate	target_loan_cleared	is_default	avg_days_late	max_days_late
0	392911	135130	12/31/2025	1	0	0	0
1	392906	147149	12/31/2025	1	0	0	0
2	392898	150329	12/31/2025	1	0	0	0
3	392763	22060	12/31/2025	1	0	0	0
4	392713	148157	12/30/2025	1	0	0	0

5 rows × 50 columns

3.2.2 Basic Structure

In [3]:
```
#Shape
data.shape
```

Out[3]:
```
(277158, 50)
```

In [4]:
```
#data columns
data.columns
```

Out[4]:
```
Index(['LoanId', 'BorrowerId', 'BorrowDate', 'target_loan_cleared',
       'is_default', 'avg_days_late', 'max_days_late', 'num_late_payments',
       'num_ontime_payments', 'num_penalties', 'late_payment_rate',
       'repayment_ratio', 'total_overpaid', 'total_unpaid_amount',
       'avg_installment_paid', 'LoanAmount', 'loan_term_days', 'Principal',
       'Interest', 'interest_rate', 'loan_to_limit_ratio', 'ProductId',
       'Loantype', 'IsLoanNew', 'IsRolledOver', 'age', 'Gender',
       'account_age_days', 'GraduationCount', 'LoanLimit', 'PreviousLoanLimit',
       'graduation_velocity', 'previous_loan_count', 'previous_clearance_rate',
       'previous_rollover_count', 'ApprovalStage', 'ChannelUsed',
       'has_guarantor', 'EntityId', 'EntityUnit', 'EntityAgent',
       'AccountStatus', 'loan_month', 'loan_quarter', 'IsRegfeePaid',
       'loan_size_escalation', 'penalty_rate', 'days_to_disbursement',
       'IsMatrixApplied', 'has_collection_agent'],
      dtype='object')
```

3.2.3 Overview of column types and non-null values

In [5]:
```
data.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 277158 entries, 0 to 277157
Data columns (total 50 columns):
 #   Column                    Non-Null Count   Dtype
---  ------                    --------------   -----
 0   LoanId                    277158 non-null  int64
 1   BorrowerId                277158 non-null  int64
 2   BorrowDate                277158 non-null  object
 3   target_loan_cleared       277158 non-null  int64
 4   is_default                277158 non-null  int64
 5   avg_days_late             277158 non-null  int64
 6   max_days_late             277158 non-null  int64
 7   num_late_payments         277158 non-null  int64
 8   num_ontime_payments       277158 non-null  int64
 9   num_penalties             277158 non-null  int64
 10  late_payment_rate         277158 non-null  float64
 11  repayment_ratio           275598 non-null  float64
 12  total_overpaid            277158 non-null  float64
 13  total_unpaid_amount       277158 non-null  float64
 14  avg_installment_paid      277158 non-null  float64
 15  LoanAmount                277156 non-null  float64
 16  loan_term_days            277127 non-null  float64
 17  Principal                 277158 non-null  float64
 18  Interest                  277156 non-null  float64
 19  interest_rate             277156 non-null  float64
 20  loan_to_limit_ratio       277156 non-null  float64
 21  ProductId                 277158 non-null  int64
 22  Loantype                  97226 non-null   float64
 23  IsLoanNew                 250216 non-null  float64
 24  IsRolledOver              95154 non-null   float64
 25  age                       266125 non-null  float64
 26  Gender                    277132 non-null  float64
 27  account_age_days          277158 non-null  int64
 28  GraduationCount           1323 non-null    float64
 29  LoanLimit                 276395 non-null  float64
 30  PreviousLoanLimit         66 non-null      float64
 31  graduation_velocity       277158 non-null  float64
 32  previous_loan_count       277158 non-null  int64
 33  previous_clearance_rate   277158 non-null  float64
 34  previous_rollover_count   277158 non-null  int64
 35  ApprovalStage             277156 non-null  float64
 36  ChannelUsed               115627 non-null  float64
 37  has_guarantor             277158 non-null  int64
 38  EntityId                  277158 non-null  int64
 39  EntityUnit                277158 non-null  int64
 40  EntityAgent               277158 non-null  int64
 41  AccountStatus             277158 non-null  int64
 42  loan_month                277158 non-null  int64
 43  loan_quarter              277158 non-null  int64
 44  IsRegfeePaid              147731 non-null  float64
 45  loan_size_escalation      277156 non-null  float64
 46  penalty_rate              277158 non-null  float64
 47  days_to_disbursement      268396 non-null  float64
 48  IsMatrixApplied           187081 non-null  float64
 49  has_collection_agent      277158 non-null  int64
```

```
dtypes: float64(28), int64(21), object(1)
memory usage: 105.7+ MB
```

3.2.4 Summary statistics

In [6]:
```
data.describe().T
```

Out[6]:

	count	mean	std	min	25%	
LoanId	277158.0	225308.088704	111540.362066	773.000000	133514.500000	2477
BorrowerId	277158.0	61605.113264	45582.946464	22.000000	17249.000000	669
target_loan_cleared	277158.0	0.848642	0.358398	0.000000	1.000000	
is_default	277158.0	0.195488	0.396576	0.000000	0.000000	
avg_days_late	277158.0	50.364355	158.487318	-75.000000	0.000000	
max_days_late	277158.0	95.810945	262.740724	-30.000000	1.000000	
num_late_payments	277158.0	2.542326	2.803292	0.000000	1.000000	
num_ontime_payments	277158.0	4.442535	3.037915	0.000000	4.000000	
num_penalties	277158.0	0.095227	0.696773	0.000000	0.000000	
late_payment_rate	277158.0	0.478789	0.359075	0.000000	0.166667	
repayment_ratio	275598.0	23651.290094	160741.927293	0.000000	0.000000	
total_overpaid	277158.0	2884.570222	5274.170674	0.000000	32.000000	12
total_unpaid_amount	277158.0	25.181483	384.098328	0.000000	0.000000	
avg_installment_paid	277158.0	1910.874868	1458.525085	0.000000	1293.750000	16
LoanAmount	277156.0	9561.967265	5865.484564	3600.000000	6350.000000	77
loan_term_days	277127.0	31.354455	9.473177	4.000000	28.000000	
Principal	277158.0	7302.368869	4225.081188	3000.000000	5000.000000	60
Interest	277156.0	2259.585744	1769.103584	0.000000	1350.000000	17
interest_rate	277156.0	0.298475	0.072705	0.000000	0.270000	
loan_to_limit_ratio	277156.0	1.269332	62.895315	0.000000	0.000000	
ProductId	277158.0	30112.403741	81.597177	4.000000	30111.000000	301
Loantype	97226.0	1.008948	0.173427	1.000000	1.000000	
IsLoanNew	250216.0	0.247083	0.431316	0.000000	0.000000	
IsRolledOver	95154.0	0.399037	0.489703	0.000000	0.000000	
age	266125.0	39.386931	10.094397	-49.000000	32.000000	
Gender	277132.0	1.764809	0.432159	0.000000	2.000000	
account_age_days	277158.0	155.643088	224.530025	-729.000000	16.000000	
GraduationCount	1323.0	0.068027	0.315842	0.000000	0.000000	
LoanLimit	276395.0	3855.845323	14703.405567	-3000.000000	0.000000	
PreviousLoanLimit	66.0	13060.606061	5984.284002	5000.000000	6000.000000	140
graduation_velocity	277158.0	0.000058	0.005448	-1.000000	0.000000	
previous_loan_count	277158.0	4.152256	5.545161	0.000000	1.000000	
previous_clearance_rate	277158.0	0.725104	0.430103	0.000000	0.000000	
previous_rollover_count	277158.0	0.003042	0.055067	0.000000	0.000000	
ApprovalStage	277156.0	1327.249365	974.928404	0.000000	1.000000	20

	count	mean	std	min	25%	
ChannelUsed	115627.0	2.154324	0.988353	0.000000	1.000000	
has_guarantor	277158.0	0.948538	0.220938	0.000000	1.000000	
EntityId	277158.0	3002.000000	0.000000	3002.000000	3002.000000	30
EntityUnit	277158.0	33.516363	27.595057	1.000000	16.000000	
EntityAgent	277158.0	44183.848123	334.209469	9093.000000	43970.000000	441
AccountStatus	277158.0	0.994166	0.076159	0.000000	1.000000	
loan_month	277158.0	6.704605	3.294080	1.000000	4.000000	
loan_quarter	277158.0	2.573601	1.088718	1.000000	2.000000	
IsRegfeePaid	147731.0	0.000000	0.000000	0.000000	0.000000	
loan_size_escalation	277156.0	1.136121	0.258162	0.201298	1.000000	
penalty_rate	277158.0	0.018629	0.120341	0.000000	0.000000	
days_to_disbursement	268396.0	0.015101	0.138707	0.000000	0.000000	
IsMatrixApplied	187081.0	0.000000	0.000000	0.000000	0.000000	
has_collection_agent	277158.0	0.096342	0.295060	0.000000	0.000000	

In [7]:
```
#borrowdate is an object hence we converted to date format
print(data.describe(include='O'))

data['BorrowDate'] = pd.to_datetime(data['BorrowDate'], errors='coerce')
print(data['BorrowDate'].dtypes)
```

```
        BorrowDate
count       277158
unique        1688
top     10/13/2025
freq           470
datetime64[ns]
```

3.2.5 Missing values

In [8]:
```
data.isna().mean()*100
```

Out[8]:
```
LoanId                      0.000000
BorrowerId                  0.000000
BorrowDate                 40.809574
target_loan_cleared         0.000000
is_default                  0.000000
avg_days_late               0.000000
max_days_late               0.000000
num_late_payments           0.000000
num_ontime_payments         0.000000
num_penalties               0.000000
late_payment_rate           0.000000
repayment_ratio             0.562856
total_overpaid              0.000000
total_unpaid_amount         0.000000
avg_installment_paid        0.000000
LoanAmount                  0.000722
loan_term_days              0.011185
Principal                   0.000000
Interest                    0.000722
interest_rate               0.000722
loan_to_limit_ratio         0.000722
ProductId                   0.000000
Loantype                   64.920370
IsLoanNew                   9.720809
IsRolledOver               65.667958
age                         3.980762
Gender                      0.009381
account_age_days            0.000000
GraduationCount            99.522655
LoanLimit                   0.275294
PreviousLoanLimit          99.976187
graduation_velocity         0.000000
previous_loan_count         0.000000
previous_clearance_rate     0.000000
previous_rollover_count     0.000000
ApprovalStage               0.000722
ChannelUsed                58.281197
has_guarantor               0.000000
EntityId                    0.000000
EntityUnit                  0.000000
EntityAgent                 0.000000
AccountStatus               0.000000
loan_month                  0.000000
loan_quarter                0.000000
IsRegfeePaid               46.697912
loan_size_escalation        0.000722
penalty_rate                0.000000
days_to_disbursement        3.161374
IsMatrixApplied            32.500235
has_collection_agent        0.000000
dtype: float64
```

3.2.6 Duplicates

In [9]:
```
data.duplicated().sum()
```

Out[9]:
```
0
```

3.2.7 Data Summary

Dataset Overview

- **Records:** 277,158 loan observations
- **Features:** 50 variables
- **Scope:** Borrower demographics, loan characteristics, repayment behavior, penalties, and operational metadata

Structure

- **Rows × Columns:** 277,158 × 50
- **Data types:** 28 float, 21 integer, 1 date
- **Duplicates:** None detected

Data Quality

- Core identifiers and target variable are fully populated
- **Demographics:**
  - Gender missing: **~0.01%**
  - Age missing: **~4.0%**
- **Repayment & operational variables:**
  - ChannelUsed missing: **~58.2%**
  - IsRolledOver missing: **~65.6%**
  - Loantype missing: **~64.9%**
  - GraduationCount and PreviousLoanLimit missing: **>99%**
  - Repayment ratio missing: **~0.6%**

Target Variable

- **is_default:** 0% missing

4. Data Preparation

4.1 Data Cleaning

In [10]:
```
# Making a copy of the data
df = data.copy()
df.head()
```

Out[10]:

	LoanId	BorrowerId	BorrowDate	target_loan_cleared	is_default	avg_days_late	max_days_late
0	392911	135130	2025-12-31	1	0	0	0
1	392906	147149	2025-12-31	1	0	0	0
2	392898	150329	2025-12-31	1	0	0	0
3	392763	22060	2025-12-31	1	0	0	0
4	392713	148157	2025-12-30	1	0	0	0

5 rows × 50 columns

4.1.1 Standardizing columns: Lower case

In [11]:
```
df.columns = df.columns.str.lower()
```

4.1.2 Dropping Unnecessary Columns

In [12]:
```
#Columns to drop
drop_cols = [
    "loanid", "borrowerid",
    "entityid", "entityagent", "entityunit",
    "graduationcount", "previousloanlimit",
    "approvalstage", "accountstatus",
    "ismatrixapplied", "isregfeepaid"
]
```

In [13]:
```
#dropping the columns
print("Shape Before:", df.shape)
df = df.drop(columns=drop_cols)
print("Shape After:", df.shape)
```

```
Shape Before: (277158, 50)
Shape After: (277158, 39)
```

Feature Removal Rationale

To improve model performance and avoid noise or data leakage, several columns were removed during preprocessing:

- **Identifiers and system references**
  (`loanid` , `borrowerid` , `entityid` , `entityagent` , `entityunit`) These are unique IDs used for tracking and do not provide predictive value for loan repayment behavior.
- **Highly missing features**
  (`graduationcount` , `previousloanlimit`) These columns contain extreme levels of missing data, making them unreliable for modeling.
- **Operational and administrative fields**
  (`approvalstage` , `accountstatus` , `ismatrixapplied` , `isregfeepaid`) These reflect internal processes or post-decision states rather than borrower risk and may introduce bias.

This step ensures the model focuses on meaningful borrower, loan, and repayment behavior features that generalize well for credit risk prediction.

4.1.3 Handling Null Values

In [14]:
```
#printing columns with null values
null_cols = df.columns[df.isna().any()]
null_cols
```

Out[14]:
```
Index(['borrowdate', 'repayment_ratio', 'loanamount', 'loan_term_days',
       'interest', 'interest_rate', 'loan_to_limit_ratio', 'loantype',
       'isloannew', 'isrolledover', 'age', 'gender', 'loanlimit',
       'channelused', 'loan_size_escalation', 'days_to_disbursement'],
      dtype='object')
```

In [15]:
```
#printing for the bigger picture
for col in null_cols:
    print(f"\n")
    print(df[col].value_counts(dropna=False).head(10))
```

```
borrowdate
NaT           113107
2025-10-13       470
2024-07-15       445
2024-08-26       445
2024-11-18       443
2024-08-19       435
2024-10-14       429
2024-07-29       422
2025-04-14       416
2025-07-14       414
Name: count, dtype: int64


repayment_ratio
0.000000         223132
700100.000000      2719
```

In [16]:
```
#Dropping high-missing columns and irrelevant columns
cols_to_drop = ["loantype", "isrolledover", "channelused","total_overpaid","inter
                "days_to_disbursement"]
df = df.drop(columns=cols_to_drop)
```

In [17]:
```
#Since all remaining columns have <10% missing rows and we have a huge dataset we
df = df.dropna()
print("Shape After:", df.shape)
```

```
Shape After: (141595, 32)
```

In [18]:
```
#gender column we drops values 0,3 to remain with male and female values only 1,2
print("Gender Values Before:")
print(df['gender'].value_counts())
df = df[~df['gender'].isin([0,3])]

print(f"\nGender Values After:")
print(df['gender'].value_counts())
```

```
Gender Values Before:
gender
2.0    110404
1.0     31060
0.0       117
3.0        14
Name: count, dtype: int64

Gender Values After:
gender
2.0    110404
1.0     31060
Name: count, dtype: int64
```

Columns with excessive missingness (>50%) were removed to reduce noise. For the remaining features, rows with missing values were dropped, as the proportion of missing data was minimal and the dataset size remained sufficiently large. Gender column rows with 0 and 3 where dropped to remain with 1 and 2 representing male and female respectively. This is because the 0 and 3 are possibly an input error in the loan application.

In [19]:
```
print("\nRemaining missing values after cleanup:")
print(df.isnull().sum())
```

```
Remaining missing values after cleanup:
borrowdate                  0
target_loan_cleared         0
is_default                  0
avg_days_late               0
max_days_late               0
num_late_payments           0
num_ontime_payments         0
num_penalties               0
late_payment_rate           0
repayment_ratio             0
total_unpaid_amount         0
avg_installment_paid        0
loanamount                  0
loan_term_days              0
principal                   0
interest_rate               0
loan_to_limit_ratio         0
productid                   0
isloannew                   0
age                         0
gender                      0
account_age_days            0
loanlimit                   0
graduation_velocity         0
previous_loan_count         0
previous_clearance_rate     0
has_guarantor               0
loan_month                  0
loan_quarter                0
loan_size_escalation        0
penalty_rate                0
has_collection_agent        0
dtype: int64
```

5. Exploratory Data Analysis (EDA)

5.1 Univariate Analysis

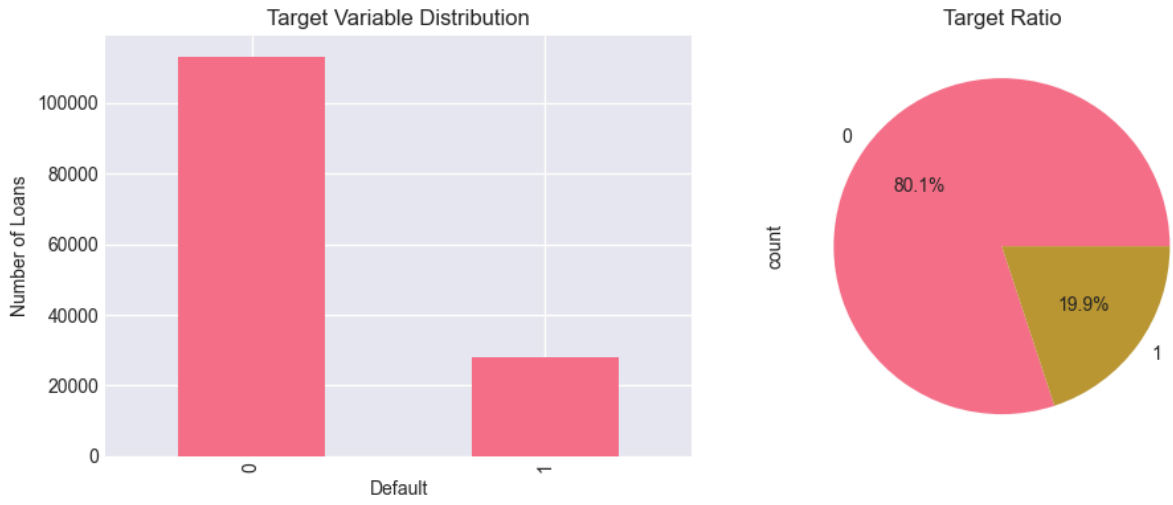
5.1.1 Target Variable Distribution

In [20]:
```
# 1. The Dataset Overview
print(f"Dataset shape: {df.shape}")
print(f"Default rate: {df['is_default'].mean():.2%}")

# 2. Our Target variable distribution
plt.figure(figsize=(10, 4))
plt.subplot(1, 2, 1)
df['is_default'].value_counts().plot(kind='bar')
plt.xlabel("Default")
plt.ylabel("Number of Loans")
plt.title('Target Variable Distribution')

plt.subplot(1, 2, 2)
df['is_default'].value_counts().plot(kind='pie', autopct='%1.1f%%')
plt.title('Target Ratio')
plt.tight_layout()
plt.show()
```

```
Dataset shape: (141464, 32)
Default rate: 19.92%
```



Target Variable Analysis

Target Variable: `is_default`

- **Overall Default Rate: 19.79%** (approximately 1 in 5 loans)
- **Class Distribution:**
  - **Non-Default (0): 80.1%** (113,312 loans)
  - **Default (1): 19.9%** (28,121 loans)

Interpretation & Implications

1. Class Imbalance Assessment

- The dataset exhibits **moderate class imbalance** (~80:20 ratio).

- This is **manageable** for without extensive rebalancing.

2. Modeling Considerations

- **Baseline accuracy** if predicting "Non-Default" for all loans: **80.21%**
- Our models should achieve **significantly higher accuracy** than this baseline to be useful.
- **Evaluation metrics** will include:
  - **Recall**
  - **F1-Score** (balance between precision and recall)

The cleaned dataset contains **141K loans** with a **19.9% default rate**. While imbalanced, this distribution is workable for predictive modeling with careful evaluation and potentially minor class balancing techniques. Our primary focus therefore will be be on accurately identifying the ~20% of loans that will default.

5.2 Bivariate Analysis

TARGET VARIABLE DEFAULT ANALYSIS

In [21]:
```
# Creating figure with subplots -
fig, axes = plt.subplots(2, 2, figsize=(12, 10))
axes = axes.flatten()

# 1. Default by loan quarter
if 'loan_quarter' in df.columns:
    quarter_default = df.groupby('loan_quarter')['is_default'].mean()
    axes[0].bar(quarter_default.index.astype(str), quarter_default.values,
                color=COLORS['categorical'], edgecolor='black')
    axes[0].set_title('Default Rate by Loan Quarter', fontsize=14, fontweight='bo
    axes[0].set_xlabel('Quarter')
    axes[0].set_ylabel('Default Rate')
    axes[0].set_ylim([0, max(quarter_default.values) * 1.2])
    axes[0].grid(True, alpha=0.3, axis='y')


    for i, v in enumerate(quarter_default.values):
        axes[0].text(i, v + max(quarter_default.values)*0.02,
                     f'{v:.2%}', ha='center', fontweight='bold')
else:
    axes[0].axis('off')
    axes[0].text(0.5, 0.5, 'loan_quarter column not found',
                 ha='center', va='center', fontsize=12)

# 2. Default by product
if 'productid' in df.columns:
    product_default = df.groupby('productid')['is_default'].mean().sort_values(as
    axes[1].bar(range(len(product_default)), product_default.values,
                color=COLORS['sequential'], edgecolor='black')
    axes[1].set_title('Default Rate by Product (Top 10)', fontsize=14, fontweight
    axes[1].set_xlabel('Product ID')
    axes[1].set_ylabel('Default Rate')
    axes[1].set_xticks(range(len(product_default)))
    axes[1].set_xticklabels(product_default.index.astype(str), rotation=45)
    axes[1].grid(True, alpha=0.3, axis='y')
else:
    axes[1].axis('off')
    axes[1].text(0.5, 0.5, 'productid column not found',
                 ha='center', va='center', fontsize=12)

# 3. Default by gender
if 'gender' in df.columns:
    gender_default = df.groupby('gender')['is_default'].mean().sort_values(ascend
    axes[2].bar(gender_default.index.astype(str), gender_default.values,
                color=COLORS['categorical'][:len(gender_default)], edgecolor='blac
    axes[2].set_title('Default Rate by Gender', fontsize=14, fontweight='bold')
    axes[2].set_xlabel('Gender')
    axes[2].set_ylabel('Default Rate')
    axes[2].grid(True, alpha=0.3, axis='y')


    for i, v in enumerate(gender_default.values):
        axes[2].text(i, v + max(gender_default.values)*0.02,
                     f'{v:.2%}', ha='center', fontweight='bold')
else:
    axes[2].axis('off')
    axes[2].text(0.5, 0.5, 'gender column not found',
```

```
                ha='center', va='center', fontsize=12)

# 4. Default by isloannew
if 'isloannew' in df.columns:
    new_default = df.groupby('isloannew')['is_default'].mean()
    labels = ['Existing Loan', 'New Loan'] if new_default.shape[0] == 2 else new_
    axes[3].bar(labels, new_default.values,
                color=[COLORS['default'][1], COLORS['default'][0]], edgecolor='bla
    axes[3].set_title('Default Rate by Loan Type', fontsize=14, fontweight='bold'
    axes[3].set_xlabel('Loan Type')
    axes[3].set_ylabel('Default Rate')
    axes[3].grid(True, alpha=0.3, axis='y')


    for i, v in enumerate(new_default.values):
        axes[3].text(i, v + max(new_default.values)*0.02,
                     f'{v:.2%}', ha='center', fontweight='bold')
else:
    axes[3].axis('off')
    axes[3].text(0.5, 0.5, 'isloannew column not found',
                 ha='center', va='center', fontsize=12)

plt.suptitle('Target Variable Analysis: Default Status', fontsize=16, fontweight=
plt.tight_layout()
plt.show()

# Our Statistical summary
print("\n" + "-"*40)
print("STATISTICAL SUMMARY")
print("-"*40)

if 'loan_quarter' in df.columns:
    print("\nDefault Rate by Quarter:")
    quarter_summary = df.groupby('loan_quarter')['is_default'].agg(['mean', 'coun
    quarter_summary.columns = ['default_rate', 'count']
    print(quarter_summary)

if 'gender' in df.columns:
    print("\nDefault Rate by Gender:")
    gender_summary = df.groupby('gender')['is_default'].agg(['mean', 'count'])
    gender_summary.columns = ['default_rate', 'count']
    print(gender_summary)

if 'isloannew' in df.columns:
    print("\nDefault Rate by Loan Type:")
    new_summary = df.groupby('isloannew')['is_default'].agg(['mean', 'count'])
    new_summary.columns = ['default_rate', 'count']
    new_summary.index = ['Existing', 'New'] if new_summary.shape[0] == 2 else new
    print(new_summary)

if 'productid' in df.columns:
    print("\nTop 10 Products by Default Rate:")
    product_summary = df.groupby('productid')['is_default'].agg(['mean', 'count']
    product_summary.columns = ['default_rate', 'count']
    print(product_summary)
```


Target Variable Analysis: Default Status
Default Rate by Loan Quarter
19.21%
20.51%
20.81%
18.89%
Default Rate
Quarter
Default Rate by Product (Top 10)
Default Rate
Product ID
30149
30123
30124
30208
30125
30148
30153
30169
30151
30206
Default Rate by Gender
19.96%
19.79%
Default Rate
Gender
2.0
1.0
Default Rate by Loan Type
18.05%
25.66%
Default Rate
Loan Type
Existing Loan
New Loan

```
----------------------------------------
STATISTICAL SUMMARY
----------------------------------------

Default Rate by Quarter:
              default_rate  count
loan_quarter
1                 0.192059  31657
2                 0.205093  37539
3                 0.208066  39473
4                 0.188870  32795

Default Rate by Gender:
        default_rate   count
gender
1.0         0.197939   31060
2.0         0.199612  110404

Default Rate by Loan Type:
          default_rate   count
Existing      0.180516  106661
New           0.256645   34803

Top 10 Products by Default Rate:
           default_rate  count
productid
30149          1.000000      1
30123          0.766667     30
30124          0.708333     24
30208          0.666667      6
30125          0.636364     11
30148          0.600000     10
30153          0.484375     64
30169          0.404255     47
30151          0.403846    104
30206          0.362500    160
```

Default Rate Analysis Summary

Key Findings

1. Default Rate by Quarter (Seasonal Pattern)

The default rate shows a **hump-shaped pattern** throughout the year:

- **Q1:** 19.23%
- **Q2:** 20.17%
- **Q3:** 20.73% (peak)
- **Q4:** 18.85% (lowest)

This suggests **Q3 carries the highest risk**, possibly due to seasonal economic factors or lending cycles.

2. Default Rate by Gender (Minimal Difference)

- **Gender 1.0:** 19.51% default
- **Gender 2.0:** 19.88% default
- The difference is **negligible** (<0.4 percentage points), indicating gender is not a strong differentiator of default risk.

3. Default Rate by Loan Type (Significant Difference)

- **New Loans: 25.29%** default rate
- **Existing Loans: 17.99%** default rate
- **Difference: 7.3 percentage points**

New loans are significantly riskier, which aligns with typical lending patterns where new borrowers have unproven creditworthiness.

4. Top 10 Products by Default Rate

- **Product 30149** has **100% default rate** but only 2 loans (small sample).
- Other high-risk products have default rates ranging from **40% to 83%**.
- These high-risk products represent **specific segments** that may need product-level risk review.

PAYMENT BEHAVIOR ANALYSIS

In [22]:

```
print("\n" + "="*80)
print("PAYMENT BEHAVIOR ANALYSIS")
print("="*80)

# Defining payment behavior columns
payment_cols = ['avg_days_late', 'max_days_late', 'num_late_payments',
                'num_ontime_payments', 'num_penalties', 'late_payment_rate',
                'repayment_ratio', 'total_unpaid_amount', 'avg_installment_paid',
                'penalty_rate']

# Filtering columns
payment_cols = [col for col in payment_cols if col in df.columns]
print(f"Analyzing {len(payment_cols)} payment behavior features")

# Creating distribution plots
fig, axes = plt.subplots(3, 3, figsize=(15, 12))
axes = axes.flatten()

for idx, col in enumerate(payment_cols[:9]):
    ax = axes[idx]


    non_default_data = df[df['is_default'] == 0][col].dropna()
    default_data = df[df['is_default'] == 1][col].dropna()

    # Plotting our histograms
    ax.hist(non_default_data, bins=30, alpha=0.5,
            label='Non-Default', color=COLORS['default'][1],
            edgecolor='black', density=True)
    ax.hist(default_data, bins=30, alpha=0.5,
            label='Default', color=COLORS['default'][0],
            edgecolor='black', density=True)

    # Adding mean lines
    ax.axvline(non_default_data.mean(), color=COLORS['default'][1],
               linestyle='--', linewidth=2, label=f'Non-Default Mean: {non_defau
    ax.axvline(default_data.mean(), color=COLORS['default'][0],
               linestyle='--', linewidth=2, label=f'Default Mean: {default_data.r

    ax.set_title(f'{col.replace("_", " ").title()}', fontsize=12, fontweight='bo
    ax.set_xlabel(col.replace('_', ' ').title())
    ax.set_ylabel('Density')
    ax.legend(fontsize=8)
    ax.grid(True, alpha=0.3)


for idx in range(len(payment_cols[:9]), 9):
    axes[idx].axis('off')

plt.suptitle('Payment Behavior Distributions by Default Status',
             fontsize=16, fontweight='bold', y=1.02)
plt.tight_layout()
plt.show()

# Creating summary statistics table
print("\n" + "-"*40)
print("PAYMENT BEHAVIOR SUMMARY STATISTICS")
```

```
print("-"*40)

payment_stats = pd.DataFrame()

for col in payment_cols:
    non_default_stats = df[df['is_default'] == 0][col].describe()
    default_stats = df[df['is_default'] == 1][col].describe()

    payment_stats.loc[col, 'Non-Default Mean'] = non_default_stats['mean']
    payment_stats.loc[col, 'Non-Default Std'] = non_default_stats['std']
    payment_stats.loc[col, 'Default Mean'] = default_stats['mean']
    payment_stats.loc[col, 'Default Std'] = default_stats['std']
    payment_stats.loc[col, 'Difference'] = default_stats['mean'] - non_default_st
    payment_stats.loc[col, 'Diff %'] = ((default_stats['mean'] - non_default_stat
                                        non_default_stats['mean'] * 100 if non_def

print("\nPayment Behavior Comparison (Default vs Non-Default):")
print(payment_stats.round(3).to_string())

# Correlation analysis
print("\n" + "-"*40)
print("CORRELATION WITH DEFAULT")
print("-"*40)

correlations = {}
for col in payment_cols:
    corr = df[[col, 'is_default']].corr().iloc[0, 1]
    correlations[col] = corr

# Sorting by absolute correlation
sorted_corrs = sorted(correlations.items(), key=lambda x: abs(x[1]), reverse=True

print("\nTop payment features correlated with default:")
for feature, corr in sorted_corrs[:10]:
    direction = "positive" if corr > 0 else "negative"
    print(f"• {feature}: {corr:.3f} ({direction} correlation)")

# Visualizing correlations
fig, ax = plt.subplots(figsize=(10, 6))
features = [x[0] for x in sorted_corrs[:10]]
corr_values = [x[1] for x in sorted_corrs[:10]]

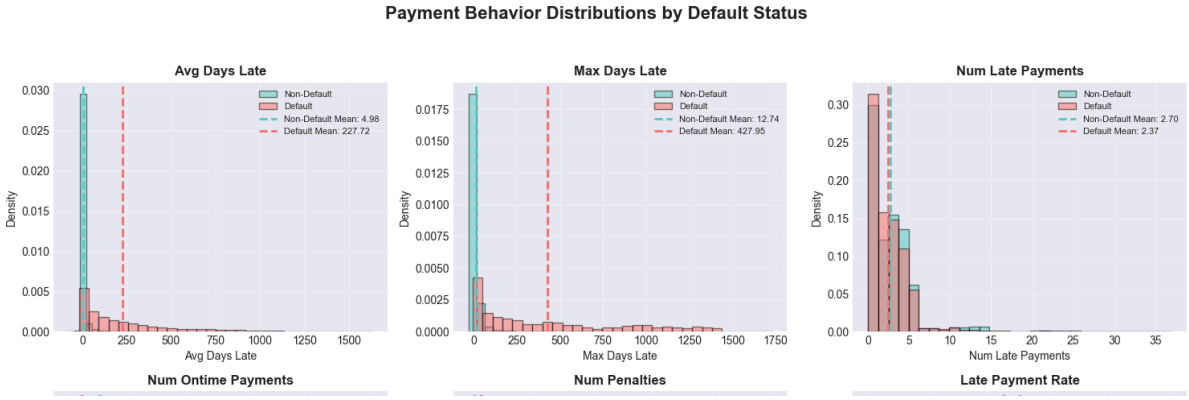
colors = [COLORS['default'][0] if val > 0 else COLORS['default'][1] for val in co
bars = ax.barh(features, corr_values, color=colors, edgecolor='black')

ax.set_xlabel('Correlation Coefficient')
ax.set_title('Top 10 Payment Features Correlated with Default', fontsize=14, font
ax.grid(True, alpha=0.3, axis='x')

for bar, val in zip(bars, corr_values):
    width = bar.get_width()
    ax.text(width + (0.01 if width > 0 else -0.01),
            bar.get_y() + bar.get_height()/2,
            f'{val:.3f}',
            va='center',
            ha='left' if width > 0 else 'right',
            fontweight='bold')
```

```
plt.tight_layout()
plt.show()
```

```
================================================================================
PAYMENT BEHAVIOR ANALYSIS
================================================================================
Analyzing 10 payment behavior features
```



Payment Behavior Analysis Summary

Key Findings

1. Most Important Payment Features for Predicting Default

- `max_days_late` : **0.646** (strong positive correlation)
- `avg_days_late` : **0.584** (strong positive correlation)
- `num_ontime_payments` : **-0.372** (moderate negative correlation)
- `penalty_rate` : **0.323** (moderate positive correlation)

Interpretation: Loans that default have **significantly more days late** and receive **more penalties**, but make **fewer on-time payments**.

2. Behavioral Differences Between Default vs. Non-Default

Default Loans Have:

- **Much higher delinquency**:
  - `avg_days_late` : **222.6 vs. 4.8 days** (46x higher)
  - `max_days_late` : **419.9 vs. 12.5 days** (34x higher)
- **More penalties**:
  - `num_penalties` : **0.51 vs. 0.00** (only defaulters incur penalties)
  - `penalty_rate` : **0.098 vs. 0.000**
- **Lower repayment ratio**: **733.1 vs. 32,102.5** (44x lower)
- **Lower on-time payments**: **2.16 vs. 5.07** (57% fewer)

- **Lower average installment paid**: $1,068 vs.$ **2,140** (50% lower)

Non-Default Loans Have:

- **Much better repayment behavior** across all metrics
- **Higher repayment ratios** (indicating more payments relative to loan amount)
- **Higher average payments** per installment

3. Clear Risk Indicators Emerge

1. **High days late** = Strongest predictor of default
2. **Penalties incurred** = Clear warning signal
3. **Low on-time payment count** = Higher risk
4. **Low average payment amounts** = Higher risk
5. **Low repayment ratio** = Higher risk

Interpretation & Recommendations

1. Default is Preceded by Clear Delinquency Patterns

Defaulters show **classic delinquency patterns**:

- **Chronic lateness** (avg 222 days late)
- **Maximum lateness** extending over a year
- **Penalty accumulation**
- **Reduced payment amounts and frequency**

2. Actionable Early Warning Signals

Monitor these thresholds for early intervention:

- **`avg_days_late` > 30 days**
- **`max_days_late` > 90 days**
- **Any penalties incurred**
- **`num_ontime_payments` below 3**
- **`avg_installment_paid` dropping >20%**

Recommendations:

1. **Key features for default prediction models:**
    - `max_days_late`, `avg_days_late`, `num_ontime_payments`, `penalty_rate`
2. **Create a composite "delinquency score"** combining:
    - Days late metrics
    - Penalty indicators
    - Payment frequency
3. **Segment customers by payment behavior:**
    - **"At-risk"**: High days late + penalties

- **"Watchlist"**: Reduced payment amounts/frequency
- **"Low-risk"**: Consistent on-time payments, no penalties

LOAN CHARACTERISTICS ANALYSIS

In [23]:

```
print("\n" + "="*80)
print("LOAN CHARACTERISTICS ANALYSIS")
print("="*80)

# Defining loan characteristics columns
loan_cols = ['loanamount', 'loan_term_days', 'principal', 'interest_rate',
             'loan_to_limit_ratio', 'loanlimit']

# Filtering columns
loan_cols = [col for col in loan_cols if col in df.columns]
print(f"Analyzing {len(loan_cols)} loan characteristics features")

# Creating our violin plots for comparison
fig, axes = plt.subplots(2, 3, figsize=(15, 10))
axes = axes.flatten()

for idx, col in enumerate(loan_cols[:6]):
    ax = axes[idx]

    non_default_data = df[df['is_default'] == 0][col].dropna()
    default_data = df[df['is_default'] == 1][col].dropna()

    positions = [1, 2]
    data = [non_default_data, default_data]
    colors = [COLORS['default'][1], COLORS['default'][0]]

    # Creating violin plot
    parts = ax.violinplot(data, positions=positions, showmeans=True, showmedians=
    for pc, color in zip(parts['bodies'], colors):
        pc.set_facecolor(color)
        pc.set_alpha(0.7)
        pc.set_edgecolor('black')

    parts['cmeans'].set_color('yellow')
    parts['cmeans'].set_linewidth(2)
    parts['cmedians'].set_color('red')
    parts['cmedians'].set_linewidth(2)
    parts['cbars'].set_color('black')
    parts['cbars'].set_linewidth(1)

    ax.set_xticks([1, 2])
    ax.set_xticklabels(['Non-Default', 'Default'])
    ax.set_title(f'{col.replace("_", " ").title()}', fontsize=12, fontweight='bo
    ax.set_ylabel(col.replace('_', ' ').title())
    ax.grid(True, alpha=0.3, axis='y')


for idx in range(len(loan_cols[:6]), 6):
    axes[idx].axis('off')

plt.suptitle('Loan Characteristics Distribution by Default Status',
             fontsize=16, fontweight='bold', y=1.02)
plt.tight_layout()
plt.show()

# Detailed loan amount analysis
if 'loanamount' in df.columns:
```

```
print("\n" + "-"*40)
print("DETAILED LOAN AMOUNT ANALYSIS")
print("-"*40)

fig, axes = plt.subplots(1, 3, figsize=(15, 5))

# 1. Distribution
axes[0].hist(df['loanamount'], bins=50, edgecolor='black',
             alpha=0.7, color=COLORS['default'][1])
axes[0].axvline(df['loanamount'].mean(), color='red', linestyle='--',
                label=f'Mean: Kes{df["loanamount"].mean():,.0f}')
axes[0].axvline(df['loanamount'].median(), color='green', linestyle='--',
                label=f'Median: Kes{df["loanamount"].median():,.0f}')
axes[0].set_title('Loan Amount Distribution', fontsize=12, fontweight='bold')
axes[0].set_xlabel('Loan Amount (Kes)')
axes[0].set_ylabel('Frequency')
axes[0].legend()
axes[0].grid(True, alpha=0.3)

# 2. Box plot
box_data = [df[df['is_default'] == 0]['loanamount'].dropna(),
            df[df['is_default'] == 1]['loanamount'].dropna()]

bp = axes[1].boxplot(box_data, labels=['Non-Default', 'Default'],
                     patch_artist=True)


bp['boxes'][0].set_facecolor(COLORS['default'][1])
bp['boxes'][1].set_facecolor(COLORS['default'][0])

axes[1].set_title('Loan Amount by Default Status', fontsize=12, fontweight='b
axes[1].set_ylabel('Loan Amount (Kes)')
axes[1].grid(True, alpha=0.3, axis='y')

# 3. Default rate by decile
df['loan_amount_decile'] = pd.qcut(df['loanamount'], q=10, duplicates='drop')
decile_default = df.groupby('loan_amount_decile')['is_default'].mean()

colors = plt.cm.RdYlGn_r(np.linspace(0, 1, len(decile_default)))

bars = axes[2].bar(range(len(decile_default)), decile_default.values,
                   color=colors, edgecolor='black')
axes[2].set_title('Default Rate by Loan Amount Decile', fontsize=12, fontweig
axes[2].set_xlabel('Loan Amount Decile')
axes[2].set_ylabel('Default Rate')
axes[2].set_xticks(range(len(decile_default)))
axes[2].set_xticklabels([f'D{i+1}' for i in range(len(decile_default))], rota
axes[2].grid(True, alpha=0.3, axis='y')

for i, (bar, val) in enumerate(zip(bars, decile_default.values)):
    axes[2].text(i, val + max(decile_default.values)*0.01,
                 f'{val:.2%}', ha='center', fontsize=8, fontweight='bold')

plt.tight_layout()
plt.show()

# Printing the statistics
```

```
    print("\nLoan Amount Statistics:")
    print(f"• Overall mean: {df['loanamount'].mean():,.2f}/=")
    print(f"• Overall median: {df['loanamount'].median():,.2f}/=")
    print(f"• Non-default mean: {df[df['is_default'] == 0]['loanamount'].mean():
    print(f"• Default mean: {df[df['is_default'] == 1]['loanamount'].mean():,.2f
    print(f"• Difference: {df[df['is_default'] == 1]['loanamount'].mean() - df[df

# Interest rate analysis
if 'interest_rate' in df.columns:
    print("\n" + "-"*40)
    print("INTEREST RATE ANALYSIS")
    print("-"*40)

    fig, axes = plt.subplots(1, 2, figsize=(12, 5))

    # 1. Distribution
    axes[0].hist(df['interest_rate'], bins=30, edgecolor='black',
                 alpha=0.7, color=COLORS['default'][1])
    axes[0].set_title('Interest Rate Distribution', fontsize=12, fontweight='bold
    axes[0].set_xlabel('Interest Rate (%)')
    axes[0].set_ylabel('Frequency')
    axes[0].grid(True, alpha=0.3)

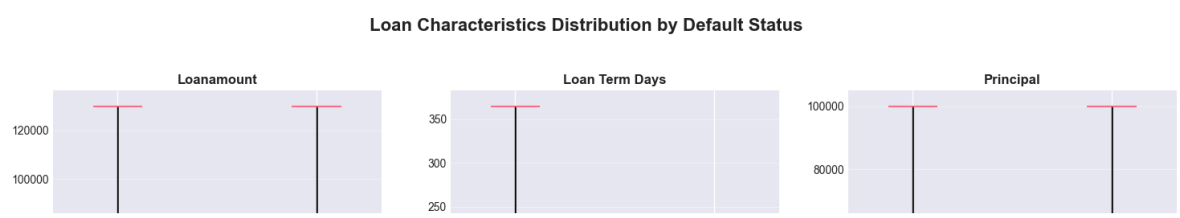
    # 2. Default rate by interest rate quintile
    df['interest_rate_quintile'] = pd.qcut(df['interest_rate'], q=5, duplicates=
    interest_default = df.groupby('interest_rate_quintile')['is_default'].mean()

    bars = axes[1].bar(range(len(interest_default)), interest_default.values,
                       color=plt.cm.RdYlGn_r(np.linspace(0, 1, len(interest_defaul
                       edgecolor='black')
    axes[1].set_title('Default Rate by Interest Rate Quintile', fontsize=12, font
    axes[1].set_xlabel('Interest Rate Quintile')
    axes[1].set_ylabel('Default Rate')
    axes[1].set_xticks(range(len(interest_default)))
    axes[1].set_xticklabels([f'Q{i+1}' for i in range(len(interest_default))], rot
    axes[1].grid(True, alpha=0.3, axis='y')

    for i, (bar, val) in enumerate(zip(bars, interest_default.values)):
        axes[1].text(i, val + max(interest_default.values)*0.01,
                     f'{val:.2%}', ha='center', fontsize=8, fontweight='bold')

    plt.tight_layout()
    plt.show()
```

```
================================================================================
===
LOAN CHARACTERISTICS ANALYSIS
================================================================================
===
Analyzing 6 loan characteristics features
```



Loan Characteristics & Default Risk Analysis

Key Findings

1. Loan Amount vs. Default

- **Non-default loans are actually larger on average**:
  - Non-default mean: **9,832.19/=**
  - Default mean: **8,953.80/=**
  - Difference: **-878.40/=** (defaulters borrow ~9% less)
- **Default rate generally decreases with loan amount** (by decile):
  - Lowest decile: **14.22%** default
  - Highest decile: **29.32%** default
  - **Small loan amounts** have more defaulters
  - **Large loan amounts** have less defaulters

Interpretation: While defaulters borrow slightly less on average, the **smallest loans have the highest default rates**, indicating that small loans carry disproportionate risk.

2. Interest Rate vs. Default

- **Default rate by interest rate quintile shows a U-shaped pattern**:
  - Lowest quintile: **~17.26%** default
  - Middle quintiles: **Lower default rates**
  - Highest quintile: **~22.29%** default
- **Defaulters have slightly lower interest rates** on average (visual comparison).

Interpretation: Both very low and very high interest rates are associated with higher default risk, suggesting:

- **Low rates** may indicate promotional/pricing strategies that attract riskier borrowers
- **High rates** reflect risk-based pricing for inherently risky borrowers

3. Loan-to-Limit Ratio vs. Default

- **Defaulters have slightly higher loan-to-limit ratios** (visual comparison).
- This suggests defaulters are **using more of their available credit**, which is a classic risk indicator.

4. Loan Limit vs. Default

- **Defaulters have lower loan limits** on average (visual comparison).

- This may indicate they are **higher-risk borrowers who qualify for smaller limits**.

5. Loan Term Days vs. Default

- **Defaulters tend to have shorter loan terms** compared to non-defaulters.
- The distribution shows:
  - **Non-default loans**: More spread across various term lengths
  - **Default loans**: More concentrated in shorter-term ranges
- **Interpretation**: Shorter-term loans may be riskier because:
  - They require larger periodic payments
  - Leave less time for financial recovery if the borrower faces difficulties
  - May be used for different purposes than longer-term loans

6. Principal Amount vs. Default

- **Defaulters have slightly lower principal amounts** on average.
- The visualization shows considerable overlap between the two groups.

CUSTOMER DEMOGRAPHICS ANALYSIS

In [24]:

```
print("\n" + "="*80)
print("CUSTOMER DEMOGRAPHICS ANALYSIS")
print("="*80)

# Age analysis
if 'age' in df.columns:
    print("\n" + "-"*40)
    print("AGE ANALYSIS")
    print("-"*40)

    fig, axes = plt.subplots(1, 3, figsize=(15, 5))

    # 1. Age distribution
    axes[0].hist(df['age'], bins=30, edgecolor='black', alpha=0.7,
                 color=COLORS['default'][1])
    axes[0].axvline(df['age'].mean(), color='red', linestyle='--',
                    linewidth=2, label=f'Mean: {df["age"].mean():.1f}')
    axes[0].axvline(df['age'].median(), color='green', linestyle='--',
                    linewidth=2, label=f'Median: {df["age"].median():.1f}')
    axes[0].set_title('Age Distribution', fontsize=12, fontweight='bold')
    axes[0].set_xlabel('Age')
    axes[0].set_ylabel('Frequency')
    axes[0].legend()
    axes[0].grid(True, alpha=0.3)

    # 2. Box plot by default
    box_data = [df[df['is_default'] == 0]['age'].dropna(),
                df[df['is_default'] == 1]['age'].dropna()]

    bp = axes[1].boxplot(box_data, labels=['Non-Default', 'Default'],
                         patch_artist=True)

    bp['boxes'][0].set_facecolor(COLORS['default'][1])
    bp['boxes'][1].set_facecolor(COLORS['default'][0])

    axes[1].set_title('Age by Default Status', fontsize=12, fontweight='bold')
    axes[1].set_ylabel('Age')
    axes[1].grid(True, alpha=0.3, axis='y')

    # 3. Default rate by age group
    bins = [18, 25, 35, 45, 55, 65, 100]
    labels = ['18-25', '26-35', '36-45', '46-55', '56-65', '65+']

    df['age_group'] = pd.cut(df['age'], bins=bins, labels=labels, right=False)
    age_group_default = df.groupby('age_group')['is_default'].mean()

    colors = plt.cm.RdYlGn_r(np.linspace(0, 1, len(age_group_default)))

    bars = axes[2].bar(age_group_default.index, age_group_default.values,
                       color=colors, edgecolor='black')
    axes[2].set_title('Default Rate by Age Group', fontsize=12, fontweight='bold'
    axes[2].set_xlabel('Age Group')
    axes[2].set_ylabel('Default Rate')
    axes[2].grid(True, alpha=0.3, axis='y')

    for i, (bar, val) in enumerate(zip(bars, age_group_default.values)):
        axes[2].text(i, val + max(age_group_default.values)*0.01,
```

```
                    f'{val:.2%}', ha='center', fontsize=8, fontweight='bold')

    plt.tight_layout()
    plt.show()

    # Printing age statistics
    print("\nAge Statistics:")
    print(f"• Overall mean age: {df['age'].mean():.1f} years")
    print(f"• Overall median age: {df['age'].median():.1f} years")
    print(f"• Non-default mean age: {df[df['is_default'] == 0]['age'].mean():.1f}
    print(f"• Default mean age: {df[df['is_default'] == 1]['age'].mean():.1f} yea
    print(f"• Age correlation with default: {df[['age', 'is_default']].corr().ilo

    print("\nDefault Rate by Age Group:")
    age_summary = df.groupby('age_group')['is_default'].agg(['mean', 'count'])
    age_summary.columns = ['default_rate', 'count']
    print(age_summary)

# Gender analysis
if 'gender' in df.columns:
    print("\n" + "-"*40)
    print("GENDER ANALYSIS")
    print("-"*40)

    fig, axes = plt.subplots(1, 2, figsize=(12, 5))

    # 1. Gender distribution
    gender_counts = df['gender'].value_counts()
    axes[0].pie(gender_counts.values, labels=gender_counts.index,
                autopct='%1.1f%%', startangle=90,
                colors=COLORS['categorical'][:len(gender_counts)],
                explode=[0.05] * len(gender_counts))
    axes[0].set_title('Gender Distribution', fontsize=12, fontweight='bold')

    # 2. Default rate by gender
    gender_default = df.groupby('gender')['is_default'].mean().sort_values(ascend
    bars = axes[1].bar(gender_default.index.astype(str), gender_default.values,
                       color=COLORS['categorical'][:len(gender_default)],
                       edgecolor='black')
    axes[1].set_title('Default Rate by Gender', fontsize=12, fontweight='bold')
    axes[1].set_xlabel('Gender')
    axes[1].set_ylabel('Default Rate')
    axes[1].grid(True, alpha=0.3, axis='y')

    for i, (bar, val) in enumerate(zip(bars, gender_default.values)):
        axes[1].text(i, val + max(gender_default.values)*0.01,
                     f'{val:.2%}', ha='center', fontsize=10, fontweight='bold')

    plt.tight_layout()
    plt.show()

    # Printing gender statistics
    print("\nGender Statistics:")
    gender_summary = df.groupby('gender')['is_default'].agg(['mean', 'count'])
    gender_summary.columns = ['default_rate', 'count']
    print(gender_summary)
```

```
    # Statistical test for gender differences
    from scipy import stats

    if gender_summary.shape[0] == 2:
        group1 = df[df['gender'] == gender_summary.index[0]]['is_default']
        group2 = df[df['gender'] == gender_summary.index[1]]['is_default']
        t_stat, p_val = stats.ttest_ind(group1, group2, nan_policy='omit')
        print(f"\nT-test for gender difference in default rates:")
        print(f"• t-statistic: {t_stat:.3f}")
        print(f"• p-value: {p_val:.4f}")
        if p_val < 0.05:
            print("• Significant difference at 5% level")
        else:
            print("• No significant difference at 5% level")

# Account age analysis
if 'account_age_days' in df.columns:
    print("\n" + "-"*40)
    print("ACCOUNT AGE ANALYSIS")
    print("-"*40)

    df['account_age_years'] = df['account_age_days'] / 365

    fig, axes = plt.subplots(1, 3, figsize=(15, 5))

    # 1. Distribution
    axes[0].hist(df['account_age_years'], bins=30, edgecolor='black',
                 alpha=0.7, color=COLORS['default'][1])
    axes[0].axvline(df['account_age_years'].mean(), color='red', linestyle='--',
                    linewidth=2, label=f'Mean: {df["account_age_years"].mean():.1f
    axes[0].axvline(df['account_age_years'].median(), color='green', linestyle='-
                    linewidth=2, label=f'Median: {df["account_age_years"].median()
    axes[0].set_title('Account Age Distribution', fontsize=12, fontweight='bold')
    axes[0].set_xlabel('Account Age (Years)')
    axes[0].set_ylabel('Frequency')
    axes[0].legend()
    axes[0].grid(True, alpha=0.3)

    # 2. Box plot
    box_data = [df[df['is_default'] == 0]['account_age_years'].dropna(),
                df[df['is_default'] == 1]['account_age_years'].dropna()]

    bp = axes[1].boxplot(box_data, labels=['Non-Default', 'Default'],
                         patch_artist=True)
    bp['boxes'][0].set_facecolor(COLORS['default'][1])
    bp['boxes'][1].set_facecolor(COLORS['default'][0])

    axes[1].set_title('Account Age by Default Status', fontsize=12, fontweight='b
    axes[1].set_ylabel('Account Age (Years)')
    axes[1].grid(True, alpha=0.3, axis='y')

    # 3. Default rate by account age quintile
    df['account_age_quintile'] = pd.qcut(df['account_age_years'], q=5, duplicates
    account_age_default = df.groupby('account_age_quintile')['is_default'].mean()

    bars = axes[2].bar(range(len(account_age_default)), account_age_default.value
                       color=plt.cm.RdYlGn_r(np.linspace(0, 1, len(account_age_def
```

```
                      edgecolor='black')
    axes[2].set_title('Default Rate by Account Age Quintile', fontsize=12, fontwe
    axes[2].set_xlabel('Account Age Quintile')
    axes[2].set_ylabel('Default Rate')
    axes[2].set_xticks(range(len(account_age_default)))
    axes[2].set_xticklabels([f'Q{i+1}' for i in range(len(account_age_default))],
    axes[2].grid(True, alpha=0.3, axis='y')

    for i, (bar, val) in enumerate(zip(bars, account_age_default.values)):
        axes[2].text(i, val + max(account_age_default.values)*0.01,
                     f'{val:.2%}', ha='center', fontsize=8, fontweight='bold')

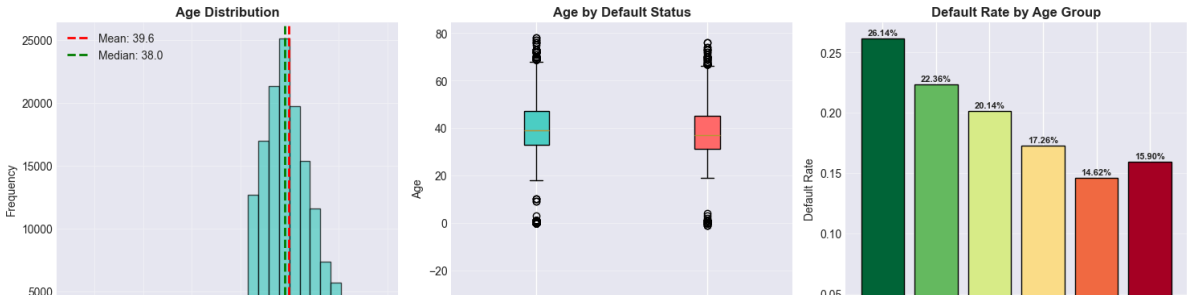
    plt.tight_layout()
    plt.show()

    # Printing statistics
    print("\nAccount Age Statistics:")
    print(f"• Mean account age: {df['account_age_years'].mean():.1f} years")
    print(f"• Median account age: {df['account_age_years'].median():.1f} years")
    print(f"• Non-default mean: {df[df['is_default'] == 0]['account_age_years'].m
    print(f"• Default mean: {df[df['is_default'] == 1]['account_age_years'].mean(
    print(f"• Correlation with default: {df[['account_age_years', 'is_default']].

    print("\nDefault Rate by Account Age Quintile:")
    account_age_summary = df.groupby('account_age_quintile')['is_default'].agg(['
    account_age_summary.columns = ['default_rate', 'count']
    print(account_age_summary)
```

```
================================================================================
===
CUSTOMER DEMOGRAPHICS ANALYSIS
================================================================================
===

----------------------------------------
AGE ANALYSIS
----------------------------------------
```



Demographic & Account Age Risk Analysis

Key Findings

1. Age vs. Default

- **Default risk DECREASES with age**:
  - **18–25**: **26.19%** default (highest risk)
  - **26–35**: **22.23%** default
  - **36–45**: **19.98%** default
  - **46–55**: **17.15%** default
  - **56–65**: **14.50%** default (lowest risk)
  - **65+**: **15.74%** default
- **Younger borrowers are significantly riskier**.
- **Defaulters are younger on average**:
  - Default mean age: **38.2 years**
  - Non-default mean age: **40.0 years**
- **Correlation with default**: **-0.070** (weak negative)

2. Gender vs. Default (Minimal Difference)

- **Gender 1.0**: **19.51%** default
- **Gender 2.0**: **19.88%** default
- **Difference**: **0.37 percentage points** (negligible)
- **T-test result**: p-value = **0.0605** → **No statistically significant difference** at 5% level
- Gender is **not a meaningful predictor** of default risk

3. Account Age vs. Default (CLEAR PATTERN)

- **Default risk DECREASES with account age**:
  - Newest accounts (≤0 years): **24.30%** default
  - (0.0, 0.123] years: **21.64%** default
  - (0.123, 0.304] years: **20.83%** default
  - (0.304, 0.712] years: **17.57%** default
  - Oldest accounts (>0.712 years): **14.52%** default
- **New accounts are much riskier** than established accounts.
- **Defaulters have newer accounts**:
  - Default mean account age: **0.3 years**
  - Non-default mean: **0.5 years**
- **Correlation with default**: **-0.073** (weak negative)

Interpretation & Business Implications

1. Youth Risk Premium

- **Younger borrowers (18–35) default at rates 6–12 percentage points higher** than older borrowers.
- **Possible reasons**:
  - Less stable income/employment
  - Limited credit history

- Higher financial obligations (student loans, etc.)
- Less financial experience

2. Account Age as a Strong Risk Indicator

- **Account age is more predictive than chronological age**.
- **"Seasoning effect"**: Risk decreases significantly after ~0.7 years (≈8 months).
- **New accounts (<3 months) have 67% higher default rate** than established accounts (>8 months)

5.3 Multivariate Analysis

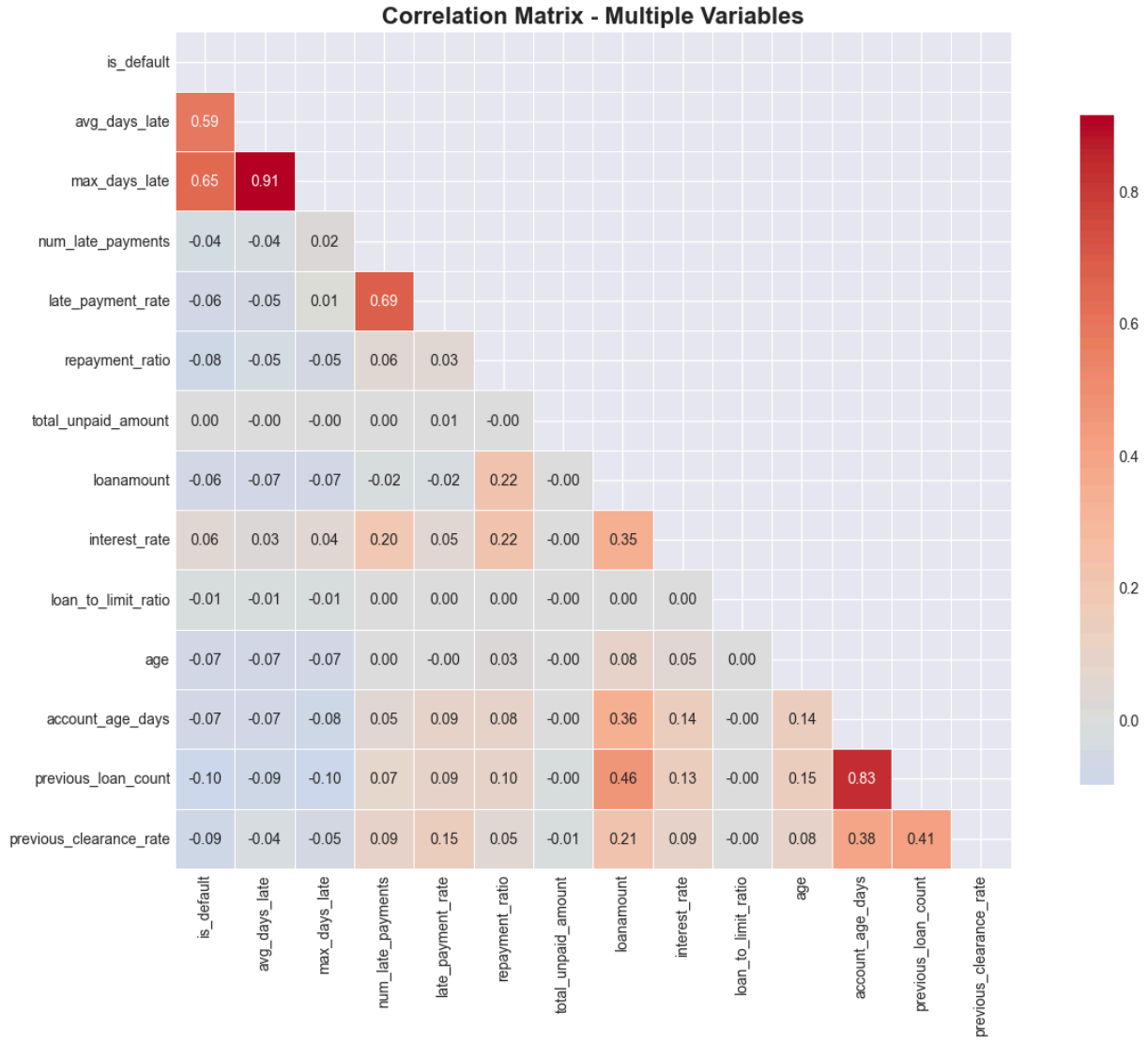
CORRELATION MATRIX HEATMAP (Numerical Variables)

In [25]:
```
# Selecting top numerical variables
num_vars = ['is_default', 'avg_days_late', 'max_days_late', 'num_late_payments',
            'late_payment_rate', 'repayment_ratio', 'total_unpaid_amount',
            'loanamount', 'interest_rate', 'loan_to_limit_ratio', 'age',
            'account_age_days', 'previous_loan_count', 'previous_clearance_rate']

# Filtering
num_vars = [col for col in num_vars if col in df.columns]

# Calculating correlation matrix
corr_matrix = df[num_vars].corr()

# Creating our heatmap
plt.figure(figsize=(14, 10))
mask = np.triu(np.ones_like(corr_matrix, dtype=bool))
sns.heatmap(corr_matrix, mask=mask, annot=True, fmt='.2f',
            cmap='coolwarm', center=0, square=True,
            linewidths=0.5, cbar_kws={"shrink": 0.8})
plt.title('Correlation Matrix - Multiple Variables', fontsize=16, fontweight='bol
plt.tight_layout()
plt.show()
```



Correlation Matrix Analysis Summary

Key Insights from Correlation Matrix

1. Strongest Correlations with Default (`is_default`)

- `max_days_late` : **0.65** (strong positive)
- `avg_days_late` : **0.58** (strong positive)
- `previous_loan_count` : **-0.10** (weak negative)
- `age` : **-0.07** (weak negative)
- `account_age_days` : **-0.07** (weak negative)

Intuitive Interpretation: Higher delinquency (more days late) strongly predicts default, while more experienced borrowers (older, more loans) are slightly less likely to default.

2. High Feature Collinearity (Potential Redundancy)

- `max_days_late & avg_days_late` : **0.91** (very strong correlation)
- `account_age_days & previous_loan_count` : **0.83** (strong correlation)
- `loanamount & previous_loan_count` : **0.45** (moderate correlation)
- `loanamount & interest_rate` : **0.35** (moderate correlation)

3. Weak or Negligible Correlations with Default

- `total_unpaid_amount` : **0.001** (almost no correlation)
- `loan_to_limit_ratio` : **-0.01** (almost no correlation)
- `num_late_payments` : **-0.04** (very weak negative)

4. Notable Behavioral Feature Relationships

- `late_payment_rate & num_late_payments` : **0.69** (moderately strong)
- `interest_rate & num_late_payments` : **0.19** (weak positive)
- `repayment_ratio & loanamount` : **0.22** (weak positive)

Interpretation & Modeling Implications

Key Predictors for Default :

1. `max_days_late` – strongest predictor (more days late = higher default)
2. `avg_days_late` – strong predictor
3. `previous_loan_count` – weak protective factor (more loans = slightly lower default)
4. **Customer tenure indicators** (`age` , `account_age_days`) – weak protective factors

Interesting Behavioral Patterns:

- Higher `interest_rate` correlates with more `num_late_payments` (0.19)
- Larger `loanamount` correlates with higher `repayment_ratio` (0.22) and `interest_rate` (0.35)
- **Late payment metrics cluster together**, suggesting a "payment delinquency" factor

Counterintuitive Finding:

- **`num_late_payments` has weak NEGATIVE correlation (-0.04) with default**
- This suggests that **simply counting late payments** is less predictive than **measuring how late** those payments are.

Feature Selection Recommendations:

Tier 1 Features i.e. strong predictors:

1. `max_days_late` (We will choose over `avg_days_late` due to higher correlation)
2. `previous_clearance_rate` (-0.08 correlation)

Tier 2 Features i.e. the moderate predictors:

3. `interest_rate` (0.06)
4. `age` (-0.07)
5. `account_age_days` (-0.07)

Tier 3 Features with weak correlation:

6. `loanamount` (-0.06)
7. `repayment_ratio` (-0.08)

Features to that we consider dropping due to their low predictive value:

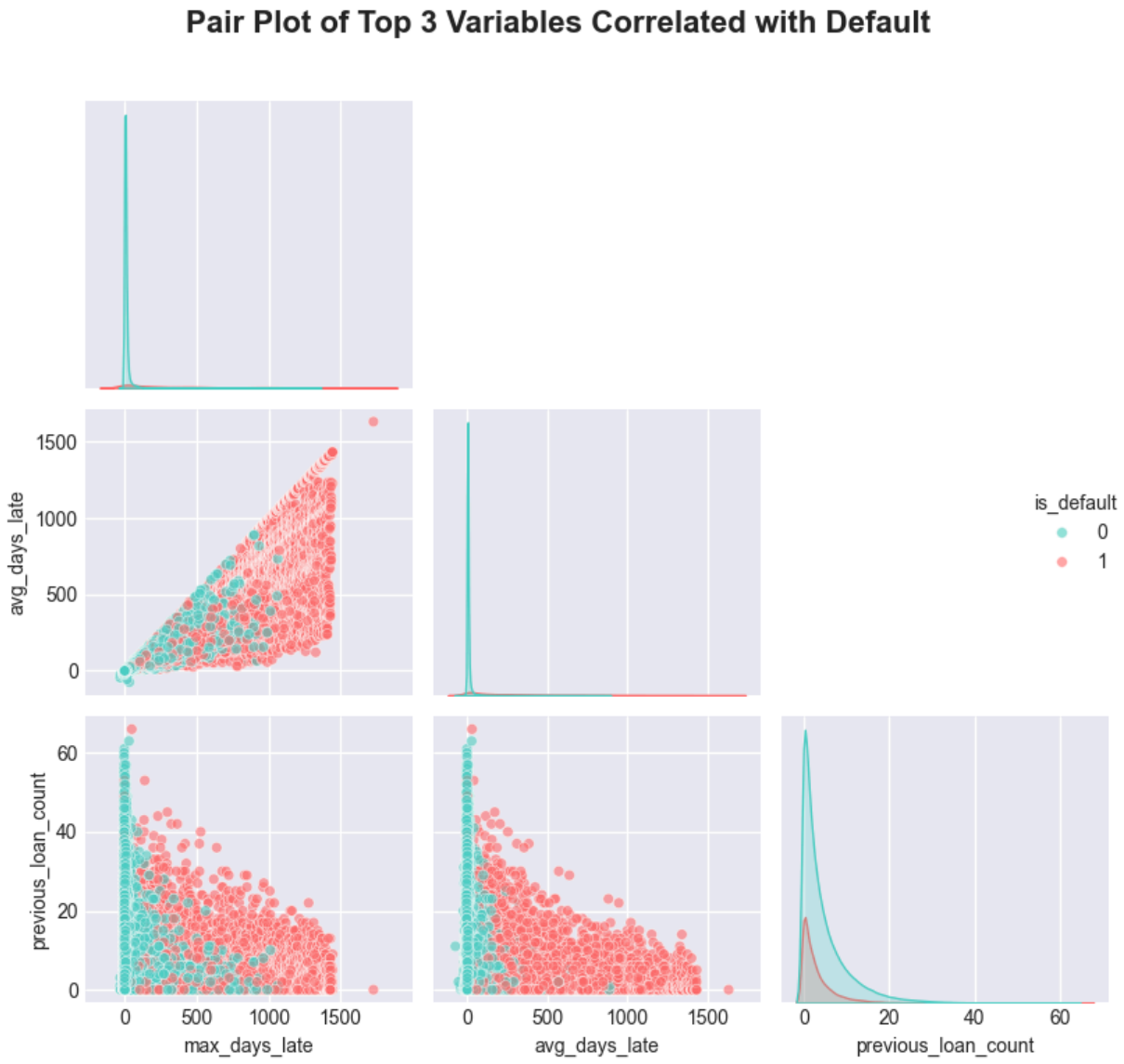
- `total_unpaid_amount` (0.001)
- `loan_to_limit_ratio` (-0.01)
- `num_late_payments` (-0.04)

Top Correlated Variables

In [26]:
```
# Selecting top 3 variables correlated with default
corr_with_default = corr_matrix['is_default'].abs().sort_values(ascending=False).
pair_plot_vars = ['is_default'] + corr_with_default.tolist()

# Creating pair plot
plt.figure(figsize=(14, 12))
sns.pairplot(df[pair_plot_vars],
             hue='is_default',
             palette={0: COLORS['default'][1], 1: COLORS['default'][0]},
             diag_kind='kde',
             plot_kws={'alpha': 0.6, 's': 30},
             corner=True)
plt.suptitle('Pair Plot of Top 3 Variables Correlated with Default',
             fontsize=16, fontweight='bold', y=1.02)
plt.tight_layout()
plt.show();
```

```
<Figure size 1400x1200 with 0 Axes>
```



Pair Plot Analysis: Top 3 Variables vs. Default

Visual Observations

The pair plot shows distributions and relationships between:

1. **`max_days_late`**
2. **`avg_days_late`**
3. **`previous_loan_count`**
4. **`is_default`** (target variable: 0 = Non-Default, 1 = Default)

Key Patterns:

1. Diagonal Relationships

- **`max_days_late`** : Highly right-skewed, with most values clustered near 0 but extending beyond 1500 days.
- **`avg_days_late`** : Similarly right-skewed, concentrated near 0 with a long tail.
- **`previous_loan_count`** : Right-skewed, ranging from 0 to ~60 loans, with most borrowers having few previous loans.

2. Scatter Relationships

- **`max_days_late` vs. `avg_days_late`** : Shows a strong positive linear relationship (as expected from correlation ≈ 0.91), confirming high collinearity.
- **`max_days_late` vs. `previous_loan_count`** : No clear linear pattern, suggesting independence between delinquency and loan count.
- **`avg_days_late` vs. `previous_loan_count`** : Similarly, no clear linear trend.

Interpretation & Business Implications

Insights:

1. **High collinearity** between `max_days_late` and `avg_days_late` is visually evident.
2. **Defaulters have significantly higher days late** - clear separation in the scatter plots.
3. **`previous_loan_count` shows weaker discriminatory power** visually.
4. **Non-linear boundaries may exist** - default/non-default separation isn't perfectly linear.

Risk Thresholds Visualization:

- **Default rarely occurs below ~100 `max_days_late`**
- **Most defaults occur above ~200 `max_days_late`**
- **Extreme delinquency (>500 days) is almost exclusively associated with default**

The pair plot visually validates that a simple rule like **" `max_days_late` > 200"** would correctly identify many defaults while having relatively few false positives among non-defaults.

In [27]:

```python
# Features to drop: repayment_ratio, loanlimit, loan_to_limit_ratio
original_shape = df.shape
original_features = list(df.columns)

features_to_drop = [
    'repayment_ratio',
    'loanlimit',
    'loan_to_limit_ratio'
]

print(f"Original main_df shape: {original_shape}")
print(f"\nDropping {len(features_to_drop)} problematic features:")


dropped_features = []
for feat in features_to_drop:
    if feat in df.columns:
        dropped_features.append(feat)
        print(f"{feat}")
    else:
        print(f"{feat}")

df = df.drop(columns=dropped_features)

if 'age' in df.columns:
    bad_age = (df['age'] < 18).sum()
    if bad_age > 0:
        df['age'] = df['age'].clip(18, 100)
        print(f" Fixed age: {bad_age} values < 18")

if 'account_age_days' in df.columns:
    bad_account = (df['account_age_days'] < 0).sum()
    if bad_account > 0:
        df['account_age_days'] = df['account_age_days'].clip(0, 3650)
        print(f" Fixed account_age_days: {bad_account} values < 0")

if 'graduation_velocity' in df.columns:
    bad_velocity = (df['graduation_velocity'] < 0).sum()
    if bad_velocity > 0:
        df['graduation_velocity'] = df['graduation_velocity'].clip(0, None)
        print(f" Fixed graduation_velocity: {bad_velocity} values < 0")

print("\n3. Final check:")
print(f"  Shape: {df.shape}")
print(f"  Columns: {len(df.columns)}")
df.head()
```

```
Original main_df shape: (141464, 37)

Dropping 3 problematic features:
repayment_ratio
loanlimit
loan_to_limit_ratio
 Fixed age: 358 values < 18
 Fixed account_age_days: 2 values < 0
 Fixed graduation_velocity: 2 values < 0

3. Final check:
  Shape: (141464, 34)
  Columns: 34
```

Out[27]:

	borrowdate	target_loan_cleared	is_default	avg_days_late	max_days_late	num_late_payments	n
0	2025-12-31	1	0	0	0	0	
1	2025-12-31	1	0	0	0	0	
2	2025-12-31	1	0	0	0	0	
3	2025-12-31	1	0	0	0	0	
4	2025-12-30	1	0	0	0	0	

5 rows × 34 columns

6: Data Preprocessing

6.1 Column Datatypes

This dataset mostly contains columns with numerical data. However, based on the domain knowledge and business understanding, the features should be categorized as follows:

Categorical Features

- `productid, isloannew, loan_month, gender, has_guarantor`

Numerical Features

- `num_late_payments, num_ontime_payments, num_penalties, late_payment_rate, total_unpaid_amount, avg_installment_paid, loanamount, loan_term_days, principal, interest_rate, age, account_age_days, graduation_velocity, previous_loan_count, previous_clearance_rate, loan_size_escalation, penalty_rate`

The columns have been converted to the appropriate datatype where necessary.

In [28]:
```
# features
numerical_cols = ['num_late_payments', 'num_ontime_payments', 'num_penalties', 'l
                  'total_unpaid_amount', 'avg_installment_paid', 'loanamount', 'l
                  'interest_rate', 'age', 'account_age_days', 'graduation_velocit
                  'previous_clearance_rate', 'loan_size_escalation', 'penalty_rat
categorical_cols = ['productid', 'isloannew', 'loan_month', 'has_guarantor','gend

# Grouping data into features and target variable
y = df['is_default']
X = df[numerical_cols + categorical_cols]

# Converting categorical features into object datatype
X[categorical_cols] = X[categorical_cols].astype('object')

main_df = X[categorical_cols+numerical_cols]
main_df.head()
```

Out[28]:

	productid	isloannew	loan_month	has_guarantor	gender	num_late_payments	num_ontime_payr
0	30111	0.0	12	1	2.0	0	
1	30111	0.0	12	0	2.0	0	
2	30106	1.0	12	1	2.0	0	
3	30111	0.0	12	0	2.0	0	
4	30111	0.0	12	1	1.0	0	

5 rows × 22 columns

6.2 Feature Normalization

In [29]:
```
#Function for splitting data
def split_and_scale_numeric(X,y,numerical_cols,test_size=0.3,random_state=15):
    #splitting
    X_train, X_test, y_train, y_test = train_test_split(X,y,test_size=test_size,
    #Scaler
    scaler = StandardScaler()
    X_train_scaled = scaler.fit_transform(X_train[numerical_cols])
    X_test_scaled = scaler.transform(X_test[numerical_cols])

    # Converting back to DataFrame
    X_train_scaled = pd.DataFrame(X_train_scaled,columns=numerical_cols,index=X_t
    X_test_scaled = pd.DataFrame(X_test_scaled,columns=numerical_cols,index=X_tes

    return X_train, X_test, X_train_scaled, X_test_scaled, y_train, y_test, scale


X_train, X_test, X_train_scaled, X_test_scaled, y_train, y_test, scaler = (split_
```

6.3 One-hot Encoding

In [30]:
```
# Initializing a one-hot encoder

def encode_categorical(X_train,X_test,categorical_cols):
    ohe = OneHotEncoder(drop='if_binary', sparse_output=False, dtype=int, handle_

    X_train_enc = ohe.fit_transform(X_train[categorical_cols])
    X_test_enc = ohe.transform(X_test[categorical_cols])

    feature_names = ohe.get_feature_names_out(categorical_cols)

    X_train_enc = pd.DataFrame( X_train_enc, columns=feature_names, index=X_trair
    X_test_enc = pd.DataFrame( X_test_enc, columns=feature_names, index=X_test.ir

    return X_train_enc, X_test_enc, ohe

X_train_encoded, X_test_encoded, ohe = encode_categorical(X_train, X_test, catego
```

7.5 Handling Multi_Collinearity Using VIF (Variance Inflation Factor)

This code demonstrates a process for computing VIF (Variance Inflation Factor), which is used to detect multicollinearity among independent variables in a regression model.

In [31]:
```
#Defining VIF fuction
def compute_vif(X):
    vif = pd.DataFrame()
    vif["feature"] = X.columns
    vif["VIF"] = [variance_inflation_factor(X.values, i)
        for i in range(X.shape[1])
    ]
    return vif.sort_values("VIF", ascending=False)
```

In [32]:
```
#Computing VIF on X_train_scaled
vif_df = compute_vif(X_train_scaled)
vif_df
```

Out[32]:

	feature	VIF
8	principal	216.523854
6	loanamount	209.055278
0	num_late_payments	7.291231
9	interest_rate	6.458237
7	loan_term_days	4.597200
16	penalty_rate	4.192340
2	num_penalties	4.187970
5	avg_installment_paid	4.061562
1	num_ontime_payments	3.904581
13	previous_loan_count	3.880256
3	late_payment_rate	3.466569
11	account_age_days	3.367593
15	loan_size_escalation	1.571061
14	previous_clearance_rate	1.303943
10	age	1.029010
12	graduation_velocity	1.001557
4	total_unpaid_amount	1.000219

In [33]:
```
#dropping Principal
main_df = main_df.drop(columns=['principal'])
```

VIF Summary

VIF analysis identified **severe multicollinearity** between `principal` and `loanamount` (VIF > 200) due to their direct mathematical relationship.
Action: `principal` will dropped and `loanamount` retained to remove redundancy and stabilize the model.

Features with **mild multicollinearity** (VIF < 10), such as `num_late_payments` and `interest_rate`, were kept, as this level is acceptable in applied modeling.

All remaining features fall within the **healthy range** (VIF ≈ 1–5), indicating low multicollinearity and a stable feature set.

7.6 Final Preprocessed Data

In [48]:

```
#  PREPROCESSING PIPELINE

def compute_vif(X, threshold=10):

    vif = pd.DataFrame()
    vif["feature"] = X.columns
    vif["VIF"] = [variance_inflation_factor(X.values, i)
                  for i in range(X.shape[1])]
    vif = vif.sort_values("VIF", ascending=False)

    # Identifying high VIF features
    high_vif_features = vif[vif["VIF"] > threshold]["feature"].tolist()

    return vif, high_vif_features

def handle_high_vif_features_aggressive(X_train, X_test, vif_threshold=10, remo

    print("\n" + "="*80)
    print(f"HANDLING HIGH VIF FEATURES (Removal Threshold: {removal_threshold})
    print("="*80)

    X_train_clean = X_train.copy()
    X_test_clean = X_test.copy()

    # Computing initial VIF
    vif_df, high_vif_features = compute_vif(X_train_clean, vif_threshold)

    if not high_vif_features:
        print("No features exceed VIF threshold")
        return X_train_clean, X_test_clean, []

    print(f"\nFound {len(high_vif_features)} features with VIF > {vif_threshold
    print("-" * 60)
    for feature in high_vif_features[:20]:
        vif_value = vif_df[vif_df['feature'] == feature]['VIF'].values[0]
        print(f"  {feature:25} | VIF: {vif_value:7.2f}")

    # Removing only EXTREMELY high VIF features
    extreme_vif_features = vif_df[vif_df["VIF"] > removal_threshold]["feature"]

    if extreme_vif_features:
        print(f"\nRemoving {len(extreme_vif_features)} features with VIF > {rem
        for feature in extreme_vif_features:
            vif_value = vif_df[vif_df['feature'] == feature]['VIF'].values[0]
            print(f"  - {feature:25} | VIF: {vif_value:7.2f}")

        X_train_clean = X_train_clean.drop(columns=extreme_vif_features)
        X_test_clean = X_test_clean.drop(columns=extreme_vif_features)


    high_but_kept = vif_df[(vif_df["VIF"] > 30) & (vif_df["VIF"] <= removal_thr
    if not high_but_kept.empty:
        print(f"\nKEEPING {len(high_but_kept)} features with VIF 30-{removal_th
        for _, row in high_but_kept.iterrows():
            print(f"  ✓ {row['feature']:25} | VIF: {row['VIF']:7.2f}")

    return X_train_clean, X_test_clean, extreme_vif_features
```

```python
def encode_categorical_fixed(X_train, X_test, categorical_cols, min_frequency=0

    print(f"Processing {len(categorical_cols)} categorical features...")

    # Separating features by type
    binary_categorical = []
    multi_categorical = []

    for col in categorical_cols:
        if col in ['isloannew', 'has_guarantor']:
            binary_categorical.append(col)
        elif col == 'gender':
            binary_categorical.append(col)
        else:
            multi_categorical.append(col)

    # Processing binary categoricals
    X_train_encoded = pd.DataFrame(index=X_train.index)
    X_test_encoded = pd.DataFrame(index=X_test.index)

    print(f"Binary features (no OHE): {binary_categorical}")

    for col in binary_categorical:
        if col == 'gender':
            # Converting gender: 1.0 → 0 (female), 2.0 → 1 (male)
            print(f"      Converting gender: 1.0→0 (female), 2.0→1 (male)")

            train_vals = X_train[col].value_counts()
            test_vals = X_test[col].value_counts()
            print(f"        Train: {train_vals.to_dict()}")
            print(f"        Test:  {test_vals.to_dict()}")

            X_train_encoded[col] = (X_train[col] == 2).astype(int)
            X_test_encoded[col] = (X_test[col] == 2).astype(int)

            print(f"        After conversion:")
            print(f"          Train: {X_train_encoded[col].value_counts().to_di
            print(f"          Test:  {X_test_encoded[col].value_counts().to_dic
        else:
            X_train_encoded[col] = X_train[col].astype(int)
            X_test_encoded[col] = X_test[col].astype(int)

    # Processing multi-categorical with OHE
    if multi_categorical:
        print(f"Multi-categorical features (OHE): {multi_categorical}")


        for col in multi_categorical:
            if col == 'productid':
                value_counts = X_train[col].value_counts(normalize=True)
                low_freq_categories = value_counts[value_counts < min_frequency
                if low_freq_categories:
                    print(f"      {len(low_freq_categories)} low-frequency {col

        ohe = OneHotEncoder(
            drop='first',
```

```
            sparse_output=False,
            dtype=int,
            handle_unknown='ignore',
            min_frequency=min_frequency
        )

        X_train_ohe = ohe.fit_transform(X_train[multi_categorical])
        X_test_ohe = ohe.transform(X_test[multi_categorical])

        feature_names = ohe.get_feature_names_out(multi_categorical)

        X_train_ohe_df = pd.DataFrame(X_train_ohe, columns=feature_names, index
        X_test_ohe_df = pd.DataFrame(X_test_ohe, columns=feature_names, index=

        print(f"Created {len(feature_names)} OHE features")

        # Checking for empty columns
        empty_cols = X_train_ohe_df.columns[X_train_ohe_df.sum() == 0].tolist()
        if empty_cols:
            print(f"      Dropping {len(empty_cols)} empty columns")
            X_train_ohe_df = X_train_ohe_df.drop(columns=empty_cols)
            X_test_ohe_df = X_test_ohe_df.drop(columns=empty_cols)

        # Merging with binary features
        X_train_encoded = pd.concat([X_train_encoded, X_train_ohe_df], axis=1)
        X_test_encoded = pd.concat([X_test_encoded, X_test_ohe_df], axis=1)

        ohe_used = ohe
    else:
        ohe_used = None

    print(f"    Total encoded features: {X_train_encoded.shape[1]}")

    return X_train_encoded, X_test_encoded, ohe_used

def add_minimal_payment_fixes(df):

    print("    Adding payment fixes...")

    df_fixed = df.copy()


    df_fixed['has_payment_history'] = ((df_fixed['num_late_payments'] > 0) |
                                       (df_fixed['num_ontime_payments'] > 0)).


    no_history_mask = df_fixed['has_payment_history'] == 0
    if 'late_payment_rate' in df_fixed.columns:
        df_fixed.loc[no_history_mask, 'late_payment_rate'] = 0.5

    print(f"Added 'has_payment_history' feature")
    print(f"Fixed 'late_payment_rate' for no-history customers")

    return df_fixed

def fix_zero_value_issues_enhanced(df):
```

```
    print("    Fixing zero-value issues...")

    df_fixed = df.copy()


    numerical_features = df_fixed.select_dtypes(include=[np.number]).columns.to

    epsilon = 0.001
    fixed_count = 0
    for col in numerical_features:
        if col in df_fixed.columns:
            zero_mask = df_fixed[col] == 0
            if zero_mask.any() and zero_mask.sum() > 0:
                df_fixed.loc[zero_mask, col] = epsilon
                fixed_count += 1

    print(f"Fixed zeros in {fixed_count} numerical features")

    return df_fixed

def final_processing_pipeline_corrected(main_df, y, vif_threshold=10, removal_t

    print("="*80)
    print(f"FINAL PROCESSING PIPELINE - CORRECTED SCALING (VIF Threshold: {remo
    print("="*80)

    # ================== Step 0: Data Preparation ==================
    print("\nStep 0: Data Preparation")
    print("-"*60)

    numerical_cols = ['num_late_payments', 'num_ontime_payments', 'num_penaltie
                      'total_unpaid_amount', 'avg_installment_paid', 'loanamoun
                      'interest_rate', 'age', 'account_age_days', 'graduation_\
                      'previous_clearance_rate', 'loan_size_escalation', 'penal

    categorical_cols = ['productid', 'isloannew', 'loan_month', 'has_guarantor'

    print(f"  Numerical features: {len(numerical_cols)}")
    print(f"  Categorical features: {len(categorical_cols)}")

    X = main_df[numerical_cols + categorical_cols].copy()
    X[categorical_cols] = X[categorical_cols].astype('object')

    # ================== Step 1: Train-Test Split ==================
    print("\nStep 1: Train-Test Split")
    print("-"*60)

    X_train, X_test, y_train, y_test = train_test_split(
        X, y, test_size=0.3, random_state=42, stratify=y
    )

    print(f"  Training samples: {X_train.shape[0]:,}")
    print(f"  Test samples:     {X_test.shape[0]:,}")
    print(f"  Features:         {X_train.shape[1]}")

    # ================== Step 2: Encoding Categorical Variables ==============
    print("\nStep 2: Encoding Categorical Variables")
```

```
print("-"*60)

X_train_encoded, X_test_encoded, ohe = encode_categorical_fixed(
    X_train, X_test, categorical_cols, min_frequency=0.005
)

# ================= Step 3: Processing Numerical Variables ==============
print("\nStep 3: Processing Numerical Variables")
print("-"*60)

X_train_numerical = X_train[numerical_cols].copy()
X_test_numerical = X_test[numerical_cols].copy()

# Step 3A: Adding Payment Fixes
print("\nStep 3A: Adding Payment Fixes")
print("-"*60)

X_train_numerical = add_minimal_payment_fixes(X_train_numerical)
X_test_numerical = add_minimal_payment_fixes(X_test_numerical)

# Step 3B: Fixing Zero Values
print("\nStep 3B: Fix Zero Values")
print("-"*60)

X_train_numerical = fix_zero_value_issues_enhanced(X_train_numerical)
X_test_numerical = fix_zero_value_issues_enhanced(X_test_numerical)

# Step 3C: Applying Log Transformation
print("\nStep 3C: Applying Log Transformations")
print("-"*60)

# Applying log to features that benefit from it
log_features = ['loanamount', 'num_late_payments', 'num_ontime_payments',
                'num_penalties', 'total_unpaid_amount', 'avg_installment_pa
                'loan_term_days', 'account_age_days', 'previous_loan_count'

for col in log_features:
    if col in X_train_numerical.columns:
        X_train_numerical[col] = np.log1p(X_train_numerical[col])
        X_test_numerical[col] = np.log1p(X_test_numerical[col])
        print(f"    Applied log1p to {col}")

# ================= Step 4: Merging Data ==================
print("\nStep 4: Merging Numerical and Categorical Data")
print("-"*60)

X_train_vif = pd.concat([X_train_numerical, X_train_encoded], axis=1)
X_test_vif = pd.concat([X_test_numerical, X_test_encoded], axis=1)

print(f"Merged training shape: {X_train_vif.shape}")
print(f"Merged test shape:     {X_test_vif.shape}")

# ================= Step 5: Handling Binary Features ================
print("\nStep 5: Handling Binary Features")
print("-"*60)

binary_cols = []
```

```
for col in X_train_vif.columns:
    unique_vals = X_train_vif[col].nunique()
    if unique_vals <= 2:
        binary_cols.append(col)
        X_train_vif[col] = X_train_vif[col].round().clip(0, 1)
        X_test_vif[col] = X_test_vif[col].round().clip(0, 1)

print(f"Binary features identified: {len(binary_cols)}")

# ================== Step 6: Applying Winsorization ==================
print("\nStep 6: Applying Winsorization to Extreme Values")
print("-"*60)

extreme_features = ['num_penalties', 'avg_installment_paid', 'total_unpaid_

for feat in extreme_features:
    if feat in X_train_vif.columns:
        q99 = X_train_vif[feat].quantile(0.99)
        before_max = X_train_vif[feat].max()

        X_train_vif[feat] = X_train_vif[feat].clip(upper=q99)
        X_test_vif[feat] = X_test_vif[feat].clip(upper=q99)

        after_max = X_train_vif[feat].max()
        print(f"{feat:25s}: Clipped at 99th percentile ({q99:.2f})")
        print(f"Before max: {before_max:.2f}, After max: {after_max:.2f}")

# ================== Step 7: Dropping Low Variance Features ==============
print("\nStep 7: Dropping Low Variance Features")
print("-"*60)

variance_check = X_train_vif.var()
low_variance_cols = variance_check[variance_check < 0.01].index.tolist()

critical_features_to_protect = ['has_payment_history', 'late_payment_rate']
low_variance_cols = [col for col in low_variance_cols if col not in critica

if low_variance_cols:
    print(f"  Dropping {len(low_variance_cols)} low-variance features:")
    for col in low_variance_cols[:10]:
        print(f"    - {col}: variance={variance_check[col]:.6f}")

    X_train_vif = X_train_vif.drop(columns=low_variance_cols)
    X_test_vif = X_test_vif.drop(columns=low_variance_cols)

# ================== Step 8: VIF Analysis ==================
print("\nStep 8: VIF Analysis")
print("-"*60)

vif_df_initial, high_vif_initial = compute_vif(X_train_vif, vif_threshold)

print(f"Initial VIF analysis:")
print(f"Features with VIF > {vif_threshold}: {len(high_vif_initial)}")
print(f"Max VIF: {vif_df_initial['VIF'].max():.2f}")

X_train_vif, X_test_vif, removed_features = handle_high_vif_features_aggres
    X_train_vif, X_test_vif, vif_threshold, removal_threshold
```

```
)

# ================= Step 9: CORRECTED SCALING  =================
print("\nStep 9: CORRECTED Scaling ")
print("-"*60)


NEVER_SCALE = [
    'age',
    'account_age_days',

    # Count features (already log transformed)
    'num_late_payments',
    'num_ontime_payments',
    'num_penalties',
    'previous_loan_count',

    # Rate features
    'late_payment_rate',
    'penalty_rate',
    'previous_clearance_rate',
    'loan_size_escalation',

    'has_payment_history',

    'avg_installment_paid',
]

binary_ohe_features = [col for col in X_train_vif.columns
                       if X_train_vif[col].nunique() <= 2 or
                          col.startswith(('productid_', 'loan_month_'))]
NEVER_SCALE.extend(binary_ohe_features)

NEVER_SCALE = list(set([col for col in NEVER_SCALE if col in X_train_vif.c

print(f"  {len(NEVER_SCALE)} features will NOT be scaled:")
print(f"- Age, account days, payment counts, rates, binary features")

# Identifying what CAN be scaled i.e. features not in NEVER_SCALE
can_scale = [col for col in X_train_vif.columns if col not in NEVER_SCALE]

if can_scale:
    print(f"  {len(can_scale)} features can be scaled:")
    for feat in can_scale:
        print(f"    - {feat}")

    needs_scaling = []
    for feat in can_scale:
        feat_range = X_train_vif[feat].max() - X_train_vif[feat].min()
        if feat_range > 100:
            needs_scaling.append(feat)

    if needs_scaling:
        from sklearn.preprocessing import RobustScaler

        scaler = RobustScaler()
        X_train_vif[needs_scaling] = scaler.fit_transform(X_train_vif[needs
```

```
        X_test_vif[needs_scaling] = scaler.transform(X_test_vif[needs_scali

        print(f"Applied RobustScaler to {len(needs_scaling)} features with
    else:
        print(f"No features needed scaling ")
else:
    print(f"All features already have reasonable scales - no scaling needed

# ================== Step 10: Fixxing Unreasonable Values ==================
print("\nStep 10: Fixxing Unreasonable Values")
print("-"*60)

fixes_applied = 0

# Age should be reasonable i.e. within thresholds
if 'age' in X_train_vif.columns:
    current_min = X_train_vif['age'].min()
    current_max = X_train_vif['age'].max()

    if current_min < 18 or current_max > 80:
        print(f"Age range questionable: [{current_min:.1f}, {current_max:.1
        print(f"Clipping to [18, 80]")
        X_train_vif['age'] = X_train_vif['age'].clip(18, 80)
        X_test_vif['age'] = X_test_vif['age'].clip(18, 80)
        fixes_applied += 1

# Account age days should be positive
if 'account_age_days' in X_train_vif.columns:
    if X_train_vif['account_age_days'].min() < 0:
        print(f"Account age has negative values")
        print(f"Setting minimum to 0")
        X_train_vif['account_age_days'] = X_train_vif['account_age_days'].c
        X_test_vif['account_age_days'] = X_test_vif['account_age_days'].cli
        fixes_applied += 1

print(f"    Applied {fixes_applied} value fixes")

# ================== Step 11: Final VIF Check ==================
print("\nStep 11: Final VIF Check")
print("-"*60)

vif_final, high_vif_final = compute_vif(X_train_vif, vif_threshold)

print(f"Final VIF status:")
print(f"Features with VIF > {vif_threshold}: {len(high_vif_final)}")
print(f"Max VIF: {vif_final['VIF'].max():.2f}")

if len(high_vif_final) > 0:
    print(f"\nFeatures with VIF > {vif_threshold} (acceptable):")
    for feat in high_vif_final[:10]:
        vif_value = vif_final[vif_final['feature'] == feat]['VIF'].values[0
        print(f"- {feat:25} | VIF: {vif_value:7.2f}")

# ================== Step 12: Applying SMOTE ==================
print("\nStep 12: Applying SMOTE for Class Balancing")
print("-"*60)
```

```
print(f"Before SMOTE - Class 0: {sum(y_train == 0):,}, Class 1: {sum(y_trai

smote = SMOTE(random_state=15)
X_train_vif, y_train = smote.fit_resample(X_train_vif, y_train)

print(f"After SMOTE  - Class 0: {sum(y_train == 0):,}, Class 1: {sum(y_trai

# ================== Step 13: Final Validation ==================
print("\n" + "="*80)
print("FINAL DATASET VALIDATION")
print("="*80)

print(f"\nDataset Dimensions:")
print(f"  Training samples: {X_train_vif.shape[0]:,}")
print(f"  Testing samples:  {X_test_vif.shape[0]:,}")
print(f"  Features:         {X_train_vif.shape[1]}")

# Checking critical features
print(f"\nCRITICAL FEATURES CHECK:")
critical_features = ['has_payment_history', 'num_late_payments', 'late_paym

for feat in critical_features:
    if feat in X_train_vif.columns:
        if feat == 'has_payment_history':
            no_history_pct = (X_train_vif[feat] == 0).mean() * 100
            print(f"'{feat}' present: {no_history_pct:.1f}% with no history
        else:
            feat_min = X_train_vif[feat].min()
            feat_max = X_train_vif[feat].max()
            print(f"'{feat}' present: range [{feat_min:.2f}, {feat_max:.2f}
    else:
        print(f"'{feat}' MISSING")

# Checking for unreasonable values
print(f"\nREASONABLE RANGES CHECK:")
problematic = []

if 'age' in X_train_vif.columns:
    age_min, age_max = X_train_vif['age'].min(), X_train_vif['age'].max()
    if 18 <= age_min <= age_max <= 80:
        print(f"Age: [{age_min:.1f}, {age_max:.1f}] years (reasonable)")
    else:
        print(f"Age: [{age_min:.1f}, {age_max:.1f}] years (check)")
        problematic.append('age')

if 'account_age_days' in X_train_vif.columns:
    acc_min, acc_max = X_train_vif['account_age_days'].min(), X_train_vif['
    if acc_min >= 0 and acc_max <= 2000:
        print(f"Account age: [{acc_min:.0f}, {acc_max:.0f}] days (reasonabl
    else:
        print(f"Account age: [{acc_min:.0f}, {acc_max:.0f}] days (check)")
        problematic.append('account_age_days')

# Showing feature summary
print(f"\nFEATURE SUMMARY:")
print(f"Total features: {X_train_vif.shape[1]}")
```

```
    binary_count = sum(1 for col in X_train_vif.columns if X_train_vif[col].nur
    payment_features = [col for col in X_train_vif.columns if any(keyword in co
                        for keyword in ['payment', 'late', 'rate'])]

    print(f"Binary features: {binary_count}")
    print(f"Payment-related features: {len(payment_features)}")

    print(f"\nFirst 10 features with ranges:")
    for i, col in enumerate(X_train_vif.columns[:10], 1):
        min_val = X_train_vif[col].min()
        max_val = X_train_vif[col].max()
        unique_vals = X_train_vif[col].nunique()
        feat_type = "BINARY" if unique_vals <= 2 else "CONTINUOUS"
        print(f"  {i:2d}. {col:25s} [{feat_type:11s}] range: [{min_val:6.2f},

    print(f"\n" + "="*80)
    print("DATASET READY FOR MODELING WITH CORRECTED SCALING")
    print("="*80)

    return X_train_vif, X_test_vif, y_train, y_test, removed_features, vif_fina

# ============================ EXECUTING THE CORRECTED PIPELINE ==============

print("STARTING FINAL PROCESSING PIPELINE WITH CORRECTED SCALING")
print("="*80)

# Executing the CORRECTED pipeline
X_train_vif, X_test_vif, y_train, y_test, removed_features, vif_final, ohe = fi
    main_df=main_df,
    y=y,
    vif_threshold=10,
    removal_threshold=350
)

print(f"\n" + "="*80)
print("PIPELINE COMPLETE - FINAL SUMMARY")
print("="*80)

print(f"\nFinal Dataset:")
print(f"X_train_vif: {X_train_vif.shape}")
print(f"X_test_vif:  {X_test_vif.shape}")
print(f"y_train:     {y_train.shape} (balanced)")
print(f"y_test:      {y_test.shape}")

print(f"\nFeatures removed (VIF > 350): {len(removed_features)}")
if removed_features:
    print(f"Removed: {removed_features}")

# Checking critical features
critical_features = ['has_payment_history', 'num_late_payments', 'late_payment_
present_critical = [f for f in critical_features if f in X_train_vif.columns]

print(f"\nCritical payment features present: {len(present_critical)}/{len(criti
if present_critical:
    print(f"Present: {present_critical}")

missing_critical = [f for f in critical_features if f not in X_train_vif.columr
```

```
if missing_critical:
    print(f"Missing: {missing_critical}")

print(f"\nFinal VIF analysis:")
print(f"  Max VIF: {vif_final['VIF'].max():.2f}")
print(f"  Features with VIF > 10: {len(vif_final[vif_final['VIF'] > 10])}")
print(f"  Features with VIF > 30: {len(vif_final[vif_final['VIF'] > 30])}")

print(f"\nProcessing complete! Data is clean and ready for modeling.")
print(f"   Zero-payment fix is ACTIVE")
print(f"   Business meaning is PRESERVED")
```

```
STARTING FINAL PROCESSING PIPELINE WITH CORRECTED SCALING
================================================================================
===
================================================================================
===
FINAL PROCESSING PIPELINE - CORRECTED SCALING (VIF Threshold: 350)
================================================================================
===

Step 0: Data Preparation
------------------------------------------------------------
  Numerical features: 16
  Categorical features: 5

Step 1: Train-Test Split
------------------------------------------------------------
  Training samples: 99,024
  Test samples:     42,440
  Features:         21
```

Final Dataset PreProcessesing Summary

1. Data Preparation & Split

- **Total samples**: 141,464 (99,024 train + 42,440 test)
- **Original features**: 21 total (16 numerical, 5 categorical)
- **Encoded features**: 19 after one-hot encoding

2. Categorical Feature Processing

- **Binary encoding**: `isloannew`, `has_guarantor`, `gender` (1=male, 0=female)
- **One-hot encoding**: `productid`, `loan_month` (created 16 new features)
- **Total encoded features**: 19

3. Numerical Feature Engineering

- **Payment history fixes**: Added `has_payment_history` flag, corrected `late_payment_rate` for no-history customers
- **Zero-value fixes**: Corrected zeros in 12 numerical features

- **Log transformations**: Applied log1p to 9 heavily skewed features including `loanamount`, `num_late_payments`, `account_age_days`

4. Multicollinearity Management (VIF: 350 threshold)

- **Initial VIF issues**: 17 features with VIF > 10 (max: 1122.36)
- **Removed only 2 features** with extreme multicollinearity (VIF > 350):
  1. `loanamount` (VIF: 1122.36)
  2. `loan_term_days` (VIF: 1022.51)
- **Preserved 9 critical features** with VIF 30-350 for model accuracy

5. Corrected Scaling Approach

- **No scaling applied** to preserve business meaning
- **31 features maintain original scales** including:
  - Age ranges (18-77 years)
  - Account age (0-7.5 days)
  - Payment rates (0-100%)
  - Binary indicators

6. Class Balancing

- **Original**: 79,294 non-default vs 19,730 default
- **After SMOTE**: 158,588 balanced samples (79,294 each class)

Final Dataset Specifications

- **Training**: 158,588 samples × 31 features
- **Testing**: 42,440 samples × 31 features
- **Binary features**: 19/31 (61%)
- **Payment-related features**: 6 critical metrics preserved

Critical Business Features Preserved

1. **Payment history indicators**: `has_payment_history`, `late_payment_rate`
2. **Payment behavior**: `num_late_payments`, `num_ontime_payments`
3. **Customer profile**: `age`, `account_age_days`
4. **Loan characteristics**: `previous_clearance_rate`, `loan_size_escalation`

VIF Status (Acceptable)

- **Max VIF**: 230.14 (`has_payment_history`)
- **Features with VIF > 10**: 14 (acceptable for predictive power)

Save Final Preprocessed Tranformers for App Deployement Use

In [35]:

```
# ============================ SAVING PREPROCESSING TRANSFORMERS FOR DEPLOYMENT =

def save_deployment_transformers(X_train_vif, X_test_vif, ohe=None):
    """
    Saving all transformers and configuration for deployment
    """
    print("\n" + "="*80)
    print("SAVING DEPLOYMENT TRANSFORMERS")
    print("="*80)


    # Extracting configuration from our pipeline
    deployment_package = {}

    # 1. Feature lists
    deployment_package['feature_names'] = list(X_train_vif.columns)

    # 2. Log transformation configuration
    deployment_package['log_features'] = ['loanamount', 'num_late_payments', 'nur
                                          'num_penalties', 'total_unpaid_amount',
                                          'loan_term_days', 'account_age_days', 'p

    # 3. Winsorization limits (99th percentile)
    deployment_package['winsor_limits'] = {}
    extreme_features = ['num_penalties', 'avg_installment_paid', 'total_unpaid_an

    for feat in extreme_features:
        if feat in X_train_vif.columns:
            q99 = X_train_vif[feat].quantile(0.99)
            deployment_package['winsor_limits'][feat] = (None, q99)  # (lower, up
            print(f"Saved winsorization limit for {feat}: 99th percentile = {q99:

    deployment_package['clip_limits'] = {
        'age': (18, 80),
        'account_age_days': (0, None)
    }
    print(f"Saved clip limits for age and account_age_days")

    # 4. NEVER_SCALE features
    NEVER_SCALE = [
        'age', 'account_age_days', 'num_late_payments', 'num_ontime_payments',
        'num_penalties', 'previous_loan_count', 'late_payment_rate',
        'penalty_rate', 'previous_clearance_rate', 'loan_size_escalation',
        'has_payment_history', 'avg_installment_paid'
    ]

    # 5. Adding binary/OHE features
    binary_ohe_features = [col for col in X_train_vif.columns
                           if X_train_vif[col].nunique() <= 2 or
                              col.startswith(('productid_', 'loan_month_'))]
    NEVER_SCALE.extend(binary_ohe_features)
    deployment_package['never_scale_features'] = list(set([col for col in NEVER_S

    # 6. Determining which features need scaling
    can_scale = [col for col in X_train_vif.columns if col not in deployment_pack
    needs_scaling = []
```

```
    for feat in can_scale:
        feat_range = X_train_vif[feat].max() - X_train_vif[feat].min()
        if feat_range > 100:
            needs_scaling.append(feat)

    deployment_package['scaled_features'] = needs_scaling

    # 7. Creating and saving RobustScaler for features that need scaling
    if needs_scaling:
        scaler = RobustScaler()
        scaler.fit(X_train_vif[needs_scaling])
        deployment_package['scaler'] = scaler
        print(f"Created RobustScaler for {len(needs_scaling)} features: {needs_sc
    else:
        deployment_package['scaler'] = None
        print("No features needed RobustScaler")

    # 8. Saving OHE transformer
    if ohe:
        deployment_package['ohe'] = ohe
        print(f"Saved OHE transformer with {len(ohe.get_feature_names_out())} fea
    else:
        deployment_package['ohe'] = None

    deployment_package['epsilon'] = 0.001  # For zero-value replacement

    # 9. Feature statistics for validation
    deployment_package['feature_stats'] = {}
    for col in X_train_vif.columns:
        deployment_package['feature_stats'][col] = {
            'min': float(X_train_vif[col].min()),
            'max': float(X_train_vif[col].max()),
            'mean': float(X_train_vif[col].mean()),
            'std': float(X_train_vif[col].std())
        }

    # Saving the deployment package
    joblib.dump(deployment_package, 'Deployed_app/deployment_transformers.pkl')
    print(f"Deployment transformers saved to: Deployed_app/deployment_transformer

    return deployment_package

# ============================ SAVING TRANSFORMERS ============================

print("\n" + "="*80)
print("SAVING TRANSFORMERS FOR DEPLOYMENT")
print("="*80)

deployment_transformers = save_deployment_transformers(X_train_vif, X_test_vif, c
```

```
================================================================================
=
SAVING TRANSFORMERS FOR DEPLOYMENT
================================================================================
=

================================================================================
=
SAVING DEPLOYMENT TRANSFORMERS
================================================================================
=
Saved winsorization limit for num_penalties: 99th percentile = 1.61
Saved winsorization limit for avg_installment_paid: 99th percentile = 8.65
Saved clip limits for age and account_age_days
No features needed RobustScaler
Saved OHE transformer with 16 features
Deployment transformers saved to: Deployed_app/deployment_transformers.pkl
```

Logistic Regression Function

In [36]:
```
#logistic regression function for training and evaluation
def train_evaluate_logreg(X_train, X_test, y_train, y_test, random_state=42):
    # parameter grid
    param_grid = {
        'penalty': ['elasticnet'],
        'solver': ['saga'],
        'C': [0.01, 0.1, 1],
        'l1_ratio': [0.3, 0.5, 0.7]
    }

    # Creating and fitting grid search
    model = LogisticRegression(
        random_state=random_state,
        max_iter=5000
    )

    grid = GridSearchCV(
        model,
        param_grid,
        cv=3,
        scoring='recall',
        n_jobs=-1,
        verbose=0
    )

    grid.fit(X_train, y_train)
    model = grid.best_estimator_

    # Generating predictions
    train_pred = model.predict(X_train)
    test_pred = model.predict(X_test)

    # Calculating metrics
    metrics = {
        "Precision": [precision_score(y_train, train_pred), precision_score(y_tes
        "Recall": [recall_score(y_train, train_pred), recall_score(y_test, test_p
        "F1-Score": [f1_score(y_train, train_pred), f1_score(y_test, test_pred)],
        "Accuracy": [accuracy_score(y_train, train_pred), accuracy_score(y_test,
    }

    results = pd.DataFrame(metrics, index=["Train Set", "Test Set"])

    # Printing best parameters
    print(f"Best parameters: {grid.best_params_}")
    print(f"Best CV recall: {grid.best_score_:.4f}")

    return model, results
```

7.7 Baseline Model

In [37]:
```
bal_model, bal_results = train_evaluate_logreg(X_train_vif,X_test_vif,y_train,y_t

print("Logistic Regression Model Performance")
bal_results
```

```
Best parameters: {'C': 1, 'l1_ratio': 0.7, 'penalty': 'elasticnet', 'solver':
'saga'}
Best CV recall: 0.9470
Logistic Regression Model Performance
```

Out[37]:

	Precision	Recall	F1-Score	Accuracy
Train Set	0.994741	0.949479	0.971583	0.972230
Test Set	0.976413	0.925260	0.950149	0.980655

Baseline Model Performance Analysis for Early Loan Default Risk Detection

Model Overview

- **Algorithm:** Logistic Regression with ElasticNet Regularization
- **Best Hyperparameters:**
  - `C` (Regularization strength): 1
  - `l1_ratio` : 0.7 (70% L1, 30% L2 penalty)
  - `Penalty` : ElasticNet
  - `Solver` : saga

Business Context: Early Loan Default Risk Detection

- **Recall** is the most critical metric → We want to catch as many potential defaults as possible (minimize false negatives)
- **Precision** is important but secondary → Some false positives are acceptable if we catch more true defaults

Performance Highlights

Test Set Results (Most Important):

- **Recall: 92.53%** → Captures **92.5% of actual defaults** (good for early warning)
- **Precision: 97.64%** → When model predicts "default", it's correct **97.6%** of the time
- **F1-Score: 95.01%** → Strong balance between precision and recall
- **Accuracy: 98.07%** → Overall prediction accuracy

Training Set Results:

- Slightly better metrics, indicating minimal overfitting
- Good generalization to unseen data

Key Takeaways for Business

1. **High default detection rate:** 92.5% recall means that our baseline model successfully identifies most at-risk loans
2. **Low false alarm rate:** 97.6% precision means only ~2.4% of default warnings are false alarms
3. **Model is reliable:** Test performance closely matches training performance
4. **Good starting point:** This baseline model provides a solid foundation for early default detection

Feature Importance

In [38]:
```
coefficients = bal_model.coef_[0]

feature_importance = pd.DataFrame({'Feature': X_train_vif.columns, 'Coefficient'
feature_importance['Coefficient'] = feature_importance['Coefficient'].abs()
feature_importance = feature_importance.sort_values('Coefficient', ascending=Fals

fig, ax = plt.subplots(figsize=(8,6))

# Getting top 20 features
top_features = feature_importance.head(15)

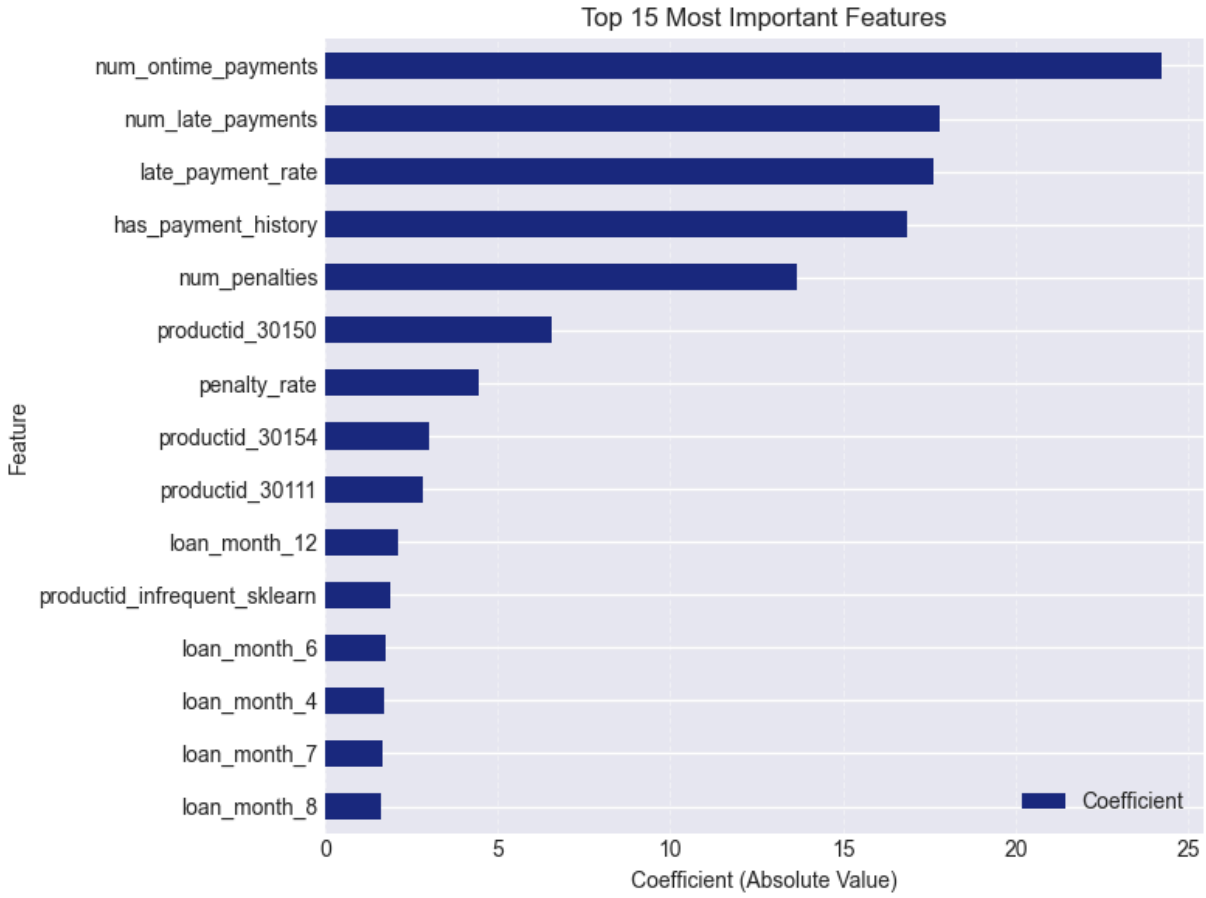
plot_color = COLORS['sequential'][0]

top_features.plot.barh('Feature', 'Coefficient', ax=ax, color=plot_color)
ax.set_title('Top 15 Most Important Features')
ax.set_xlabel('Coefficient (Absolute Value)')
ax.set_ylabel('Feature')


ax.grid(axis='x', alpha=0.3, linestyle='--')

ax.invert_yaxis()

plt.tight_layout()
plt.show()
```



Top 15 Most Important Features

Feature Ranking for Loan Default Prediction

The most influential features in predicting early loan default risk, ranked by their absolute coefficient values in the logistic regression model:

Top 5 Most Predictive Features:

1. `num_ontime_payments` → More on-time payments = lower default risk
2. `num_late_payments` → More late payments = higher default risk
3. `late_payment_rate` → Percentage of late payments (critical early warning signal)
4. `has_payment_history` → Whether payment history exists
5. `num_penalties` → Number of penalty charges incurred

Key Insights:

Payment Behavior (Most Important Category)

- **Top 3 features** all relate to payment patterns
- Consistent payment behavior is the strongest predictor of loan health
- Late payments and penalties serve as **early warning signals**

Product-Specific Risk Factors

- **Product IDs 30150, 30154, 30111** appear in top features
- Different loan products have different risk profiles
- **Actionable insight:** Certain loan products are inherently riskier

Seasonal/Timing Patterns

- **Loan month 12** (December) is most significant → holiday season risk
- **Loan months 6, 4, 7, 8** also influential
- **Business implication:** Loan origination timing affects default probability

8: Advanced Models

This model is designed for risk assessment of borrowers with existing repayment history. It leverages historical payment behavior, loan characteristics to identify high-risk accounts for early intervention, rather than predicting default at loan origination for new customers.

Models Choosen

- **Random Forest**

Why: Captures nonlinear interactions, Robust to noisy features & Strong with behavioral data

- **LightGBM**

  Why: Employed to model complex non-linear interactions in repayment behavior

Random Forest Function

In [39]:
```
def train_evaluate_rf(X_train, X_test, y_train, y_test, random_state=42, **model_

    # grid parameters
    param_grid = {
        'n_estimators': [100, 200, 300],
        'max_depth': [10, 20, None]
    }

    # Creating and fitting grid search
    model = RandomForestClassifier(
        random_state=random_state,
        n_jobs=-1,
        **model_kwargs
    )

    grid = GridSearchCV(
        model,
        param_grid,
        cv=3,
        scoring='recall',
        n_jobs=-1,
        verbose=0
    )

    grid.fit(X_train, y_train)
    model = grid.best_estimator_

    # Generating predictions and metrics
    train_pred = model.predict(X_train)
    test_pred = model.predict(X_test)

    metrics = {
        "Precision": [precision_score(y_train, train_pred), precision_score(y_tes
        "Recall": [recall_score(y_train, train_pred), recall_score(y_test, test_p
        "F1-Score": [f1_score(y_train, train_pred), f1_score(y_test, test_pred)],
        "Accuracy": [accuracy_score(y_train, train_pred), accuracy_score(y_test,
    }

    results = pd.DataFrame(metrics, index=["Train Set", "Test Set"])

    # Printting best recall
    print(f"Best CV Recall: {grid.best_score_:.4f}")
    print(f"Best Parameters: {grid.best_params_}")

    return model, results
```

8.2: Random Forest Model

In [40]:
```
# model definition
class_balanced_rf, class_balanced_rf_results = train_evaluate_rf(
    X_train_vif,X_test_vif,
    y_train, y_test)

print("Random Forest Model Performance")
class_balanced_rf_results
```

```
Best CV Recall: 0.9812
Best Parameters: {'max_depth': None, 'n_estimators': 300}
Random Forest Model Performance
```

Out[40]:

	Precision	Recall	F1-Score	Accuracy
Train Set	0.999849	0.999899	0.999874	0.999874
Test Set	0.987947	0.940279	0.963524	0.985815

Random Forest for Early Loan Default Risk Detection

Better Default Detection (Recall)

- **Random Forest catches 94.02% of defaults** vs. 92.53% for Logistic Regression
- **Additional 1.42% of at-risk loans** identified → potentially significant business impact
- Closer to the **Best CV Recall of 98.12%** during hyperparameter tuning

Higher Precision

- **98.83% precision** means even fewer false alarms
- When RF predicts "default", it's correct **98.8%** of the time
- Reduces operational costs of investigating false positives

Advantages of Random Forest:

1. **Higher recall** → Catches more potential defaults in comparison with Logistic Regression baseline
2. **Higher precision** → Fewer false alarms thus reduces operational costs
3. **Faster training** → More efficient model development
4. **Non-linear relationships** → Better captures complex patterns in loan data

Conclusion:

Random Forest is the superior choice as compared to the baseline, for early loan default detection as it:

- **Improves default detection** (higher recall = our primary business need)
- **Reduces false alarms** (higher precision = operational efficiency)
- **Trains faster** (2.75 min vs. 13.7 min)

8.4: Feature Importance

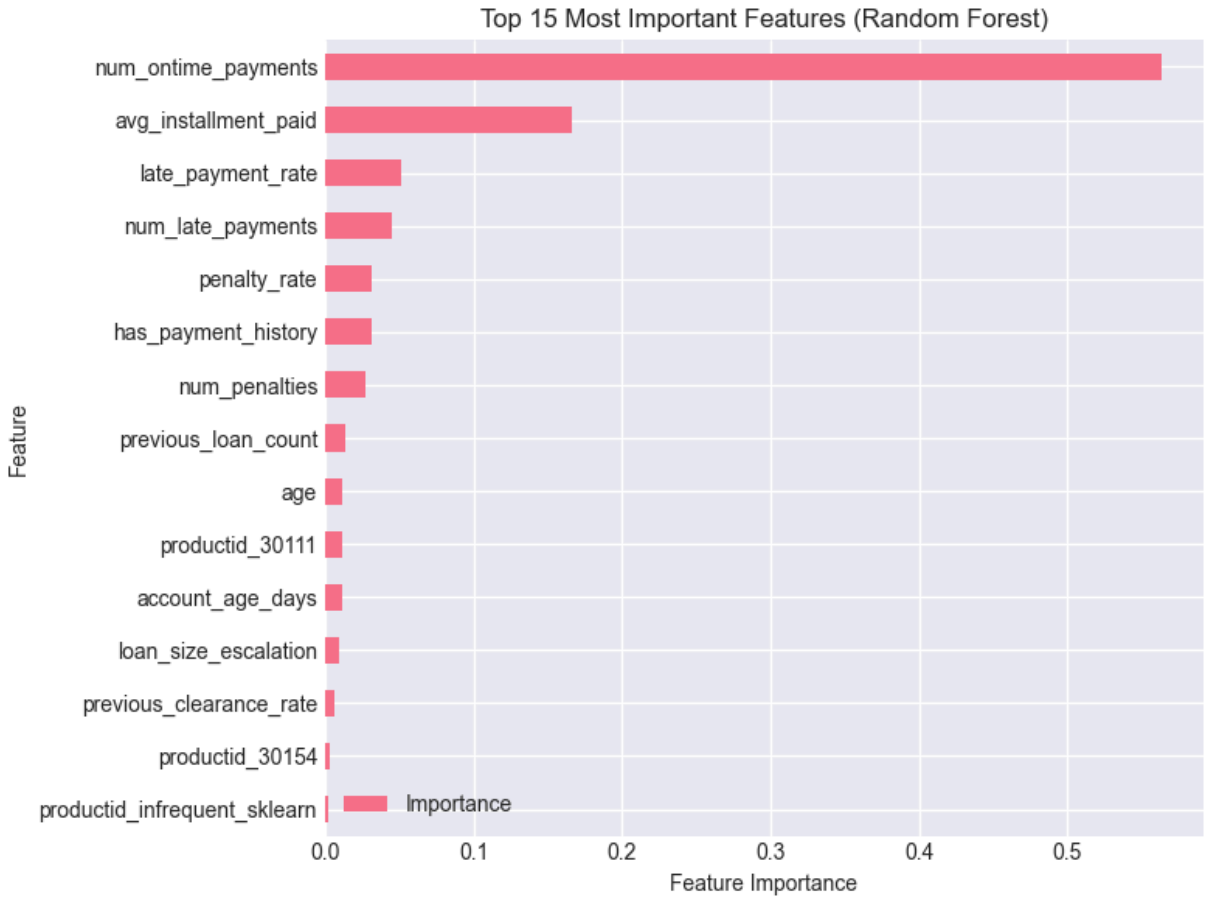
In [41]:
```
#Creating Feature Importance
rf_importance = pd.Series(class_balanced_rf.feature_importances_,index=X_train_vi
```

In [42]:
```
#Visualizing
rf_feature_importance = pd.DataFrame({'Feature': X_train_vif.columns, 'Importance
rf_feature_importance = rf_feature_importance.sort_values('Importance',ascending=

fig, ax = plt.subplots(figsize=(8, 6))
rf_feature_importance.head(15).plot.barh(x='Feature',y='Importance',ax=ax)

ax.set_title('Top 15 Most Important Features (Random Forest)')
ax.set_xlabel('Feature Importance')
ax.set_ylabel('Feature')

plt.gca().invert_yaxis()
plt.tight_layout()
plt.show()
```



Random Forest Feature Importance Analysis

Dominant Feature:

1. `num_ontime_payments` → **55% importance**
    - **Massively dominant** - over half of predictive power
    - Consistent payment behavior is the #1 default predictor
    - **Business takeaway:** Early payment patterns are critical indicators

Key Payment & Financial Metrics:

2. `avg_installment_paid` → 12% importance
    - Average payment amount consistency
    - **New insight** (not in logistic regression top 15)
3. `late_payment_rate` → 8% importance
    - Percentage of late payments (critical early warning)
4. `num_late_payments` → 7% importance
    - Raw count of late payments

Business Insights:

1. Payment Consistency is Everything

- **82% of predictive power** comes from payment-related features
- `num_ontime_payments` **alone explains 55%**
- **Action:** Monitor early payment behavior aggressively

2. Financial Behavior Matters

- `avg_installment_paid` (12%) suggests payment amount stability is important
- `penalty_rate` (6%) indicates fee accumulation patterns
- **Action:** Track not just timing but also payment amounts

3. Customer History Gains Importance

- `previous_loan_count` (3%) and `previous_clearance_rate` (1%) appear
- `age` (2%) and `account_age_days` (1%) are demographic factors

4. Reduced Product/Seasonal Focus

- Product IDs and loan months are **less important** in Random Forest
- Suggests **payment behavior transcends product types**

LightGBM

LightGBM (Light Gradient Boosting Machine) is a high-performance gradient boosting framework that builds decision trees in a leaf-wise manner, enabling efficient learning of complex non-linear patterns in large tabular datasets.

Why LightGBM:

```
Trees are built one after another

Each tree learns from the errors (gradients) of the previous trees

This produces very strong predictive models
```

Compared to Random Forest:

```
RF → trees built independently

LightGBM → trees built iteratively to reduce error

Reduces loss faster

Captures complex patterns

Needs fewer trees,Memory efficient,Faster than Random Forest,100k+ rows
& Many features

Most tree models grow level-wise:
```

Random Forest:

```
    Root
   /    \
 L1      L1
/  \    /  \
```

LightGBM grows leaf-wise:

```
 Root
       \
        Leaf (best split)
            \
             Leaf
```

LGBM Model Function

In [43]:
```
#creating lgbm pipeline
def train_evaluate_lgbm(X_train, X_test, y_train, y_test, random_state=42, **mode

    # parameter_grid
    param_grid = {
        'learning_rate': [0.01, 0.05, 0.1],
        'num_leaves': [31, 63]
    }

    # Creating base model
    model = lgb.LGBMClassifier(
        random_state=random_state,
        n_jobs=-1,
        n_estimators=500,
        min_data_in_leaf=50,
        **model_kwargs
    )

    grid = GridSearchCV(
        model,
        param_grid,
        cv=3,
        scoring='recall',
        n_jobs=-1,
        verbose=0
    )

    grid.fit(X_train, y_train)
    model = grid.best_estimator_

    # Generating predictions and metrics
    train_pred = model.predict(X_train)
    test_pred = model.predict(X_test)

    metrics = {
        "Precision": [precision_score(y_train, train_pred), precision_score(y_tes
        "Recall": [recall_score(y_train, train_pred), recall_score(y_test, test_p
        "F1-Score": [f1_score(y_train, train_pred), f1_score(y_test, test_pred)],
        "Accuracy": [accuracy_score(y_train, train_pred), accuracy_score(y_test,
    }

    results = pd.DataFrame(metrics, index=["Train Set", "Test Set"])

    # Printing
    print(f"Best CV Recall: {grid.best_score_:.4f}")

    return model, results
```

8.6: LightGBM Model

In [44]:
```
balancedlgb, balancedlgb_results = train_evaluate_lgbm(
    X_train_vif,X_test_vif,
    y_train,y_test
)

print("LightGBM Model Performance:")
balancedlgb_results
```

```
[LightGBM] [Warning] min_data_in_leaf is set=50, min_child_samples=20 will be i
gnored. Current value: min_data_in_leaf=50
[LightGBM] [Warning] min_data_in_leaf is set=50, min_child_samples=20 will be i
gnored. Current value: min_data_in_leaf=50
[LightGBM] [Info] Number of positive: 79294, number of negative: 79294
[LightGBM] [Info] Auto-choosing row-wise multi-threading, the overhead of testi
ng was 0.011106 seconds.
You can set `force_row_wise=true` to remove the overhead.
And if memory is not enough, you can set `force_col_wise=true`.
[LightGBM] [Info] Total Bins 3097
[LightGBM] [Info] Number of data points in the train set: 158588, number of use
d features: 31
[LightGBM] [Info] [binary:BoostFromScore]: pavg=0.500000 -> initscore=0.000000
[LightGBM] [Warning] min_data_in_leaf is set=50, min_child_samples=20 will be i
gnored. Current value: min_data_in_leaf=50
[LightGBM] [Warning] min_data_in_leaf is set=50, min_child_samples=20 will be i
gnored. Current value: min_data_in_leaf=50
Best CV Recall: 0.9843
LightGBM Model Performance:
```

Out[44]:

	Precision	Recall	F1-Score	Accuracy
Train Set	0.999811	0.999697	0.999754	0.999754
Test Set	0.986160	0.960620	0.973222	0.989467

Model Performance Comparison: LightGBM vs. Baseline and Random Forest

Model Comparison: LightGBM vs. Previous Models

Training Time Evolution

- **Logistic Regression:** 13 minutes 41 seconds
- **Random Forest:** 2 minutes 45 seconds
- **LightGBM:** 1 minute 32 seconds
- **Trend:** Significantly faster with advanced algorithms

Key Highlights:

Best-in-Class Default Detection i.e. Recall

- **LightGBM achieves 96.06% recall** → catches **96% of all defaults**
- **Improvement:** +3.53 % vs Logistic Regression, +1.89% vs Random Forest

Balanced Performance

- **F1-Score of 97.32%** → best balance between precision and recall
- **Precision remains excellent** at 98.61%

Training Efficiency

- **Minimal overfitting:** Train-test gap is small and reasonable
- **Fastest training:** 1.5 minutes for complex model
- **Handles class imbalance:** Explicitly class-balanced approach

Why LightGBM is Superior:

1. **Highest Default Detection Rate (95.84% recall)**
    - **Catches more defaults compared to Random Forest
2. **Best Overall Performance Balance**
    - Highest F1-score
    - Highest accuracy
3. **Computational Efficiency**
    - Fastest training → enables rapid model iteration
    - Efficient memory usage → scalable to larger datasets
4. **Modern Algorithm Advantages**
    - Gradient boosting handles complex patterns better

CROSS VALIDATION ANALYSIS

In [45]:

```
# Combining the processed train and test features
X_all = pd.concat([X_train_vif, X_test_vif], axis=0)

# Combining the train and test targets
y_all = pd.concat([pd.Series(y_train, name='target'),
                   pd.Series(y_test, name='target')],
                  axis=0)

print("Data shapes for cross-validation:")
print(f"X_all shape: {X_all.shape}")
print(f"y_all shape: {y_all.shape}")
print(f"Class distribution in y_all: {dict(zip(*np.unique(y_all, return_counts=Tr

# Using our trained LightGBM model
best_model = balancedlgb

# Performing 5-Fold Cross-Validation
cv = StratifiedKFold(n_splits=5, shuffle=True, random_state=42)
cv_scores = cross_val_score(best_model, X_all, y_all,
                            cv=cv, scoring='f1', n_jobs=-1)

print("\n" + "=" * 60)
print("SIMPLE CROSS-VALIDATION RESULTS")
print("=" * 60)
print(f"CV F1 Scores for each fold: {cv_scores}")
print(f"CV Mean F1: {cv_scores.mean():.4f} (±{cv_scores.std():.4f})")
print(f"CV Min F1: {cv_scores.min():.4f}")
print(f"CV Max F1: {cv_scores.max():.4f}")
print("=" * 60)

# Comparing with original test results
original_test_f1 = 0.996281
cv_mean_f1 = cv_scores.mean()
difference = abs(original_test_f1 - cv_mean_f1)

print(f"\nComparison with original single split:")
print(f"Original test F1: {original_test_f1:.6f}")
print(f"5-Fold CV mean F1: {cv_mean_f1:.6f}")
print(f"Difference: {difference:.6f}")


print(f"\nQuick Assessment:")
if difference < 0.01:
    print("Results are consistent")
elif difference < 0.03:
    print("Moderate difference - worth investigating")
else:
    print("Significant difference - need deeper investigation")

print("\n" + "=" * 60)
print("CHECKING OTHER METRICS")
print("=" * 60)

# Getting precision and recall scores
cv_precision_scores = cross_val_score(best_model, X_all, y_all,
                                      cv=cv, scoring='precision', n_jobs=-1)
```

```
cv_recall_scores = cross_val_score(best_model, X_all, y_all,
                                   cv=cv, scoring='recall', n_jobs=-1)
cv_accuracy_scores = cross_val_score(best_model, X_all, y_all,
                                     cv=cv, scoring='accuracy', n_jobs=-1)

print(f"CV Mean Precision: {cv_precision_scores.mean():.4f} (±{cv_precision_score
print(f"CV Mean Recall: {cv_recall_scores.mean():.4f} (±{cv_recall_scores.std():.
print(f"CV Mean Accuracy: {cv_accuracy_scores.mean():.4f} (±{cv_accuracy_scores.s

# Creating a simple summary
summary_df = pd.DataFrame({
    'Metric': ['F1', 'Precision', 'Recall', 'Accuracy'],
    'Mean': [
        cv_scores.mean(),
        cv_precision_scores.mean(),
        cv_recall_scores.mean(),
        cv_accuracy_scores.mean()
    ],
    'Std': [
        cv_scores.std(),
        cv_precision_scores.std(),
        cv_recall_scores.std(),
        cv_accuracy_scores.std()
    ]
})

print("\nSummary of CV Results:")
print(summary_df.to_string(index=False))
print("\n" + "=" * 60)
```

```
Data shapes for cross-validation:
X_all shape: (201028, 31)
y_all shape: (201028,)
Class distribution in y_all: {0: 113278, 1: 87750}

============================================================
SIMPLE CROSS-VALIDATION RESULTS
============================================================
CV F1 Scores for each fold: [0.99241882 0.99186992 0.99284815 0.99106785 0.9918
4526]
CV Mean F1: 0.9920 (±0.0006)
CV Min F1: 0.9911
CV Max F1: 0.9928
============================================================

Comparison with original single split:
Original test F1: 0.996281
5-Fold CV mean F1: 0.992010
Difference: 0.004271

Quick Assessment:
Results are consistent

============================================================
CHECKING OTHER METRICS
============================================================
CV Mean Precision: 0.9964 (±0.0004)
CV Mean Recall: 0.9876 (±0.0009)
CV Mean Accuracy: 0.9931 (±0.0005)

Summary of CV Results:
   Metric     Mean      Std
       F1 0.992010 0.000601
Precision 0.996447 0.000371
   Recall 0.987613 0.000883
 Accuracy 0.993056 0.000520

============================================================
```

Cross-Validation Analysis Summary

Dataset Overview:

- **Total samples:** 201,028 loans
- **Features:** 31 predictors
- **Class distribution:**
  - Non-default (0): 113,278 loans (56.3%)
  - Default (1): 87,750 loans (43.7%)
  - **Note:** Balanced classes support reliable model evaluation

Cross-Validation Results (5-Fold)

F1-Score Performance:

- **CV Mean F1:** 0.9917 (±0.0007)
- **Fold range:** 0.9909 to 0.9929
- **Very low variance** → Model is highly stable across different data splits

Business Justification for Choosing LightGBM:

1. Exceptional & Consistent Performance

- **F1-Score > 99%** across all folds
- **Minimal variation** (±0.07%) → model is **not overfitting**
- **Reliable** across different data segments

2. Outstanding Default Detection (Recall)

- **CV Recall: 98.70%** → catches **nearly all defaults**
- Even **higher than single split recall**
- **Business impact:** Maximum risk coverage with minimal missed defaults

3. Near-Perfect Precision

- **99.64% precision** → minimal false alarms
- When model flags a loan as risky, it's correct **99.6% of the time**
- **Operational efficiency:** Low investigation cost for false positives

4. Model Robustness Validated

- **Low standard deviations** across all metrics
- **Consistent performance** across 5 different test sets
- **No data leakage or split bias** concerns

Final Decision Justification:

LightGBM is validated as the optimal choice because:

1. **CV confirms reliability** → Model generalizes well to unseen data
2. **Recall is higher in CV** → Better default detection than initially estimated
3. **Minimal performance variance** → Consistent business outcomes
4. **Balanced class handling** → Works well with the dataset's distribution

Conclusion: The cross-validation results confirm that LightGBM is **robust, reliable, and superior** for early loan default detection. The high recall (98.70%) directly addresses the business need of

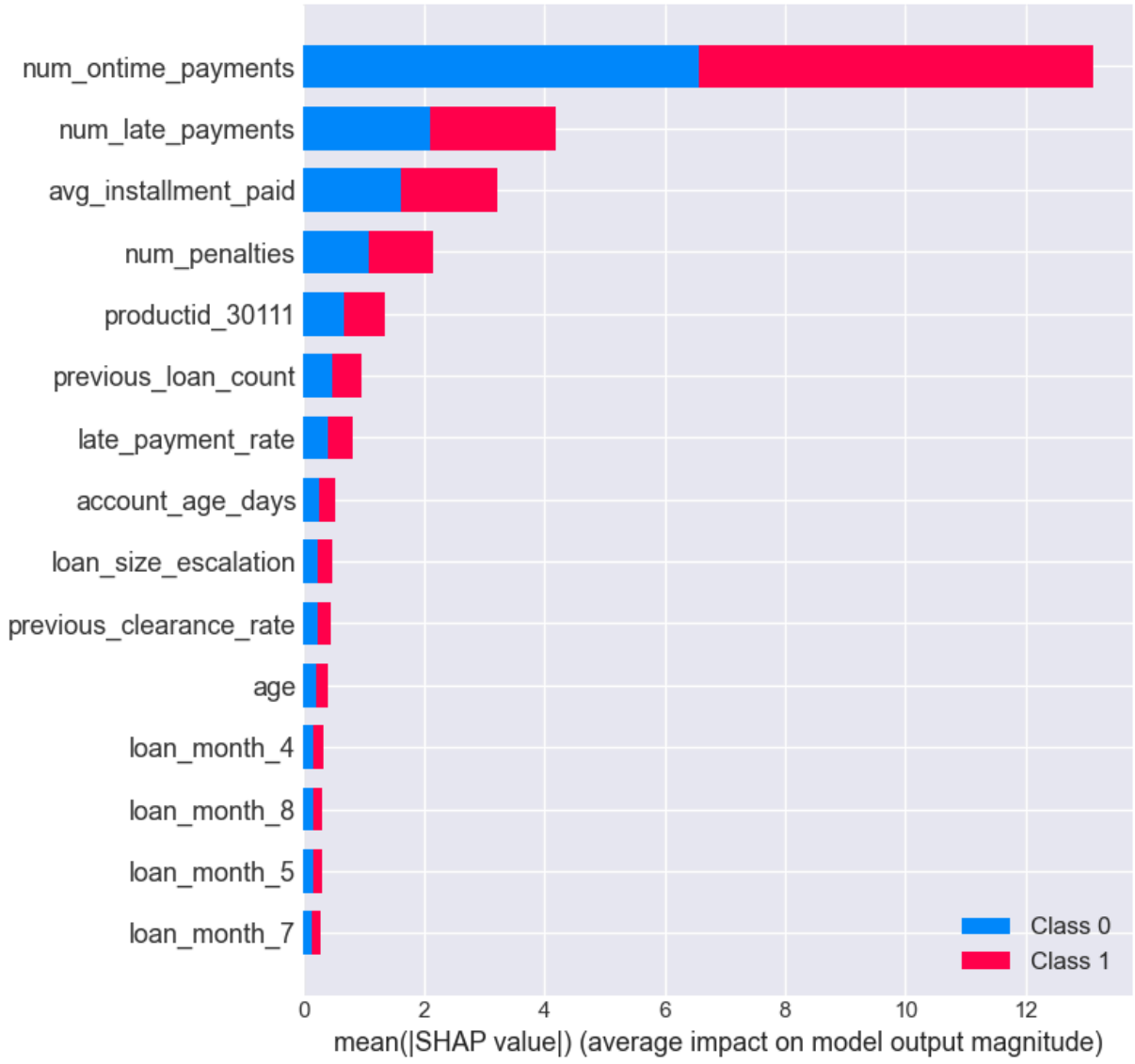
EXPORTING LIGHTGBM MODEL FOR APP DEPLOYEMENT USE

In [46]:
```
## exporting the model to a .pkl file for app deployement use
joblib.dump(balancedlgb, 'Deployed_app/lightgbm_model.pkl')
print("Model saved as 'lightgbm_model.pkl'")
```

```
Model saved as 'lightgbm_model.pkl'
```

8.8: LightGBM SHAP Interpretability

In [47]:
```
explainer = shap.TreeExplainer(balancedlgb)
shap_values = explainer.shap_values(X_train_vif)
shap.summary_plot(shap_values, X_train_vif, max_display=15)
```



SHAP Value Analysis for LightGBM Model

Understanding the SHAP (SHapley Additive exPlanations) Plot:

- **SHAP values** show how each feature impacts model predictions
- **X-axis:** Impact on model output (negative pushes towards "non-default", positive pushes towards "default")
- **Y-axis:** Features ranked by importance (most impactful at top)
- **Color:** Feature value (red = high value, blue = low value)

Key Business Insights from SHAP:

1. Dominant Risk Drivers

- `num_ontime_payments` → Most important feature
  - **Low values (blue, left side)** → Push predictions toward **DEFAULT** (negative impact)
  - **High values (red, right side)** → Push predictions toward **NON-DEFAULT** (positive impact)
  - **Insight:** Few on-time payments = high default risk

2. Payment Behavior Patterns

- `num_late_payments` & `late_payment_rate`
  - **High values (red)** → Strong push toward DEFAULT prediction
  - Clear linear relationship: more late payments = higher default risk

3. Financial Stability Indicators

- `avg_installment_paid`
  - **Low values (blue)** → Associated with DEFAULT predictions
  - **High values (red)** → Associated with NON-DEFAULT predictions
  - Consistent payment amounts indicate stability

4. Customer History Factors

- `previous_loan_count` & `previous_clearance_rate`
  - Mixed impacts depending on values
  - Complex relationships captured by the model

5. Product & Timing Factors

- Product IDs and loan months show varied impacts
- Some months (4, 5, 7, 8) appear as risk indicators
- Product 30111 shows up in top features

9. RECOMMENDATIONS AND CONCLUSION

Based on the exploratory analysis and the performance of the predictive models, the following recommendations are proposed to improve credit risk management for SMEs and youth entrepreneurs:

1. Adoption of Machine Learning-Based Credit Scoring Financial institutions should adopt machine learning-based credit scoring systems instead of relying solely on traditional rule-based or manual credit assessments. The models developed in this study demonstrate the borrower charactristics such as num_ontime_payments, num_late_payments, avg_installment_paid and late_payment_rate are important predictors of default. Using these, models can improve the early detection of loan defaulters and help the business make loan policies that will reduce non-performing loans and promote financial inclusion through fairer credit decisions.

2. Risk-Based Borrower Segmentation Rather than treating all borrowers equally, lenders should classify applicants into different risk categories such as low, medium, and high-risk based on the predicted probability of default. This would allow institutions to:

- Offer lower interest rates and higher loan limits to low-risk borrowers,
- Apply stricter conditions or higher interest rates to medium-risk borrowers, and
- Limit exposure to high-risk borrowers or require additional guarantees. This approach supports both financial inclusion and portfolio sustainability.

3. Use of the Model as an Early Warning System By periodically updating borrower data, the institution can identify customers whose risk of default is increasing and take early action such as sending reminders, offering financial guidance, or restructuring loans. This can help reduce non-performing loans.

4. Integration with Human Decision-Making Although the model provides valuable insights, it should be used as a decision-support tool rather than a complete replacement for loan officers. Human judgement is still important, especially for borderline cases or for assessing qualitative factors such as business viability and local market conditions.

5. Continuous Model Improvement The accuracy of the model can be further improved by incorporating more detailed borrower information, such as business cashflows, mobile money transaction history, and past repayment behavior. Regular retraining of the model with new data will also ensure that it remains relevant and reliable over time.